Optimizing embedded TLS client performance

by

Ganyu Xu

A thesis
presented to the University of Waterloo
in fulfillment of the
thesis requirement for the degree of
Master of Applied Science
in
Electrical and Computer Engineering

Waterloo, Ontario, Canada, 2025

Examining Committee Membership

The following served on the Examining Committee for this thesis. The decision of the Examining Committee is by majority vote.

External Examiner:	Bruce Bruce Professor, Dept. of Philosophy of Zoology, University of Wallamaloo
Supervisor(s):	Ann Elk Professor, Dept. of Zoology, University of Waterloo Andrea Anaconda Professor Emeritus, Dept. of Zoology, University of Waterloo
Internal Member:	Pamela Python Professor, Dept. of Zoology, University of Waterloo
Internal-External Member:	Meta Meta Professor, Dept. of Philosophy, University of Waterloo
Other Member(s):	Leeping Fang Professor, Dept. of Fine Art, University of Waterloo

Author's Declaration

I hereby declare that I am the sole author of this thesis. This is a true copy of the thesis, including any required final revisions, as accepted by my examiners.

I understand that my thesis may be made electronically available to the public.

Abstract

Transport Layer Security (TLS) is the most widely used cryptographic protocol on the Internet. It integrates tightly with the transport layer of the network (TCP) and ensures confidentiality, integrity, and authenticity of the data transmitted at application data layer. TLS achieves these security properties via using a combination of cryptographic primitives including Diffie-Hellman key exchange, digital signatures, cryptographic hash functions, and authenticated encryption schemes. The public-key cryptographic primitives based on number theories and elliptic curves are at the risk of becoming insecure due to advances in quantum computing algorithms and the advent of large quantum computers, which demands an urgent transition to post-quantum cryptography.

Public experimentation with transitioning to a post-quantum TLS began as early as 2016 with Google's CECPQ1. Through nearly a decade of collaboration among industry, academia, and governments, the transition to a post-quantum TLS is well underway. There is an abundance of literature and technical resources that explored the performance and security impact of replacing Diffie-Hellman key exchange with post-quantum KEMs and replacing RSA/ECDSA/EdDSA with post-quatnum digital signatures. Several groups have also proposed protocol-level optimzation to partially mitigate the performance penalty from post-quantum primitives. However, deploying PQ-TLS to embedded clients received comparatively less attention: the performance impact is less well-understood, and there is less existing resources for evaluating PQ-TLS on various embedded platforms.

In this thesis, we implemented post-quantum TLS on top of WolfSSL using PQC implementations from the PQClean project, then tested the performacne of TLS handshake with a Raspberry Pi Pico 2 W as client and a desktop server. In our testing, we found lattice-based KEM and digital signatures to perform competitively with elliptic-curve Diffie-Hellman and digital signatures despite having larger communication costs.

We also proposed and implemented two performance optimization for post-quantum TLS. First is using IND-1CCA KEM for the key exchange phase, for which we proposed an original KEM transformation that converts a passively secure PKE into an IND-1CCA KEM, which we call the "encrypt-then-MAC" transformation. Compared to the Fujisaki-Okamoto transformation, our "encrypt-then-MAC" transformation removes the need for re-encryption in decapsulation, which translates to more than 70% reduction in CPU cycles when applied to the CPA subroutines of ML-KEM. Second, we implemented KEMTLS on top of WolfSSL, which uses KEM instead of digital signature for authentication. In our testing we found KEM-based authentication to have comparable performance with signature-based authentication.

Acknowledgements

I would like to thank all the little people who made this thesis possible.

Dedication

This is dedicated to the one I love.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

Transport Layer Security (TLS) is the most widely used cryptographic protocol on the Internet. It allows two communicating parties to establish a secure channel that achieves confidentiality, integrity, and authenticity over an insecure or even hostile network. *Secure Socket Layer* (SSL), the predecessor of TLS, was originally designed by Netscape for web browsing (HTTPS), but through more than thirty years of iterations, TLS/SSL has become the default choice for many secure applications, including email (via SMTPS, IMAPS, and POP3S), remote desktop (RDP), sensor networks (MQTT/MQTT-SN), etc.

To achieve its security goals, TLS uses many cryptographic primitives including symmetric ciphers, cryptographic hash functions, Diffie-Hellman key exchange, and digital signatures. Most of these primitives are affected by advances in quantum computing such as Grover's search algorithm [51], with the public-key cryptographic components (Diffie-Hellman key exchange and digital signatures) considered "broken" because of Peter Shor's quantum algorithm [91, 92] for solving integer factorization and discrete log problem with polynomial time complexity.

Given widespread adoption of online communication for sensitive information ranging from personal banking to national defense, the threat of large-scale quantum computer demands a seismic shift toward post-quantum cryptography (PQC). The need for PQC is particularly urgent for two reasons: first, developing new cryptographic primitives and protocols requires extensive research efforts and thorough testing before they are considered ready for production deployment; second, secure communication can be intercepted today and its encryption broken some time later when large quantum computers are feasible, the so-called "harvest-now-decrypt-later" attack.

Public experimentation with post-quantum TLS began as early as 2016, when Google

implemented hybrid key exchange (X25519 + NewHope) in TLS 1.2. This was followed up by further experiments in 2018 by Google and CloudFlare implementing X25519+NTRU-HRSS and X25519+SIKE in TLS 1.3. Since then, ongoing collaboration among governments, academia, and industry produced an abundance of literature reviewing the implementation, performance, and security of adopting PQC in TLS [**?**]. As of 2025, post-quantum TLS has been widely rolled out into the consumer space, including browser support for ML-KEM shipped in Chrome, FireFox, and Safari.

While the transition to post-quantum TLS is gathering momentum, there is comparatively less attention to its impact on IoT devices. Internet-enabled embedded devices saw increased deployment in a huge variety of use cases including autonomous vehicles, medical sensors, and smart appliances. Many IoT devices also use TLS for secure communication, but post-quantum TLS brings a lot of performance penalty that degrade the quality of service from these devices because computing capacity is constrained, unique deployment challenges such as being battery operated and possibly hostile environments (secrets in firmare may be trivially dumped, etc.)

1.1 Contributions

	OT-ML-KEM-512	**OT-ML-KEM-768**	**OT-ML-KEM-1024**
Encapsulation	93157 (+1.8%)	146405 (+7.3%)	205763 (+3.3%)
Decapsulation	33733 (-72.2%)	43315 (-76.8%)	51375 (-79.1%)

Table 1.1: OT-ML-KEM performance compared to ML-KEM

Implementation	**PQC**	**Certificates**	**Client**	**Server**	**Protocol**
[24, 23]	OQS	OpenSSL	mbedTLS	mbedTLS	TLS 1.2
[106]	OQS	OpenSSL	WolfSSL	WolfSSL	TLS 1.3
[49]	OQS	Python	WolfSSL	rustls	TLS 1.3, KEMTLS
this work	PQClean	WolfSSL	WolfSSL	WolfSSL	TLS 1.3, KEMTLS

Table 1.2: Software dependencies for PQ-TLS/KEMTLS implementations

Our contributions are as follows:

- We propose "encrypt-then-MAC", a KEM transformation that converts an OW-CPA PKE into an IND-1CCA secure KEM. We instantiated this transformation

with the CPA sub-routines of ML-KEM and named it one-time ML-KEM (OT-ML-KEM). Compared to ML-KEM, OT-ML-KEM trades small performance penalty in encapsulation for up to 70% CPU cycle reduction in decapsulation (Table 1.1)

- We implemented and benchmarked post-quantum TLS 1.3 and KEMTLS on top of WolfSSL. To our knowledge, this is the first implementation where certificate generation, client (embedded and desktop), and server are all implemented on the same TLS/SSL library, thus reducing the complexity of software dependencies and facilitating future evaluation of PQ-TLS/KEMTLS on embedded or desktop environments (Table 1.2).

1.2 Related works

To our knowledge, Burstinghaus-Steinbach et al. published the first earliest implementation and evaluation of post-quantum TLS on embedded client [24, 23]. This implementation integrated Kyber and SPHINCS+ into TLS 1.2 using mbedTLS, and the test platforms include Raspberry Pi 3B, ESP32-PICO, Fieldbus Option Card, and LPCXpresso. Tasopoulos et al. [105, 106] published a follow-up implementation, this time integrating an expanded selection of NIST PQC Round 3 and 4 candidates into TLS 1.3 on top of WolfSSL. The target platform is a NUCLEO-F439ZI (ARM Cortex-M4). Schoffel et al. [87] and later Tasopoulos et al. [102, 103] also evaluated the energy consumption of post-quantum TLS on battery-powered IoT device, proving the viability of PQC on extremely low-powered devices. Last but not least, Gonzalez and Wiggers [49] implemented KEMTLS on WolfSSL and compared performance with post-quantum TLS 1.3, where they found KEMTLS to have significant performance advantage on narrowband edge devices.

The rest of this paper is organized accordingly:

- Chapter 2 reviews TLS/KEMTLS handshake protocol and their security properties.
- Chapter 3 introduces the concept of IND-1CCA security, proposes an original IND-1CCA transformation, and discusses its application to TLS key exchange
- Chapter 4 describes how we implemented post-quantum TLS and KEMTLS on an embedded client using WolfSSL
- Chapter 5 presents performance evaluation of post-quantum TLS on an embedded client

- Chapter 6 summarizes all presented results and discusses open problems and future directions

Chapter 2

An overview of TLS

Transport Layer Security (TLS) is a communication protocol with which two parties can exchange data in the presence of passive or active adversaries while preserving **confidentiality, integrity, and authenticity**. It was first developed at Netscape in 1994 and went through many iterations. The latest revision is TLS 1.3, formally standardized by IETF in RFC 8446 [84] in 2018. TLS 1.3 made significant improvements over TLS 1.2 [31] and prior versions, including deprecating weak symmetric cipher suites, requiring forward secrecy at all times, and restructuring the handshake state machine in order to simplify the handshake workflow. According to Cloudflare [110], TLS 1.3 reached more than 93% adoption on the Internet. Given TLS 1.3's superior design and nearly universal adoption over other versions of TLS, this work will focus on this version unless otherwise specified.

Section 2.1 reviews the TLS 1.3 handshake protocol, including its security properties, how they are affected by quantum computers, and the existing solutions. Section 2.2 introduces KEMTLS [88], a proposal to use KEM-based authentication in TLS 1.3. Section **??** reviews prior works on adopting PQC for TLS.

2.1 TLS 1.3 handshake

Figure 2.1 illustrates a simple TLS 1.3 handshake over TCP. A new TLS 1.3 connection begins with a handshake in which the two parties exchange a specific sequence of messages to negotiate cryptographic parameters, establish shared secrets, and authenticate each other. If the handshake is successful, then the connection transitions to exchanging application data using the encryption scheme and secret keys obtained from the handshake phase. In TLS 1.3, the handshake can be further divided into two phases:

1. **Key exchange**

   (a) `ClientHello`: client publishes its supported symmetric cipher suites (AEAD, hash functions), signature algorithms, a random nonce, and key exchange public keys.

   (b) `ServerHello`: server chooses its preferred symmetric cipher suite, as well as publishes its own random nonce and key exchange public keys.

2. **Authentication**

   (a) `Certificates`: server sends a chain of X.509 certificates that bind its identity to a digital signature public key; optionally, server can request client authentication (`CertificateRequest`), in which case the client must also send a chain of certificates.

   (b) `CertificatesVerify`: server proves ownership fo the corresponding digital signature secret key by producing a signature over the handshake transcript; client will do the same upon server's request

   (c) `Finished`: both parties prove to each other the integrity of the handshake by producing an HMAC over the handshake transcript

2.1.1 Certificate-based authentication

Authentication in TLS 1.3 relies on digital signatures and X.509 certificates. Figure 2.2 illustrates a certificate chain.

Each X.509 certificate encodes three pieces of information: identities of the issuer and the subject, a cryptographic public key, and a cryptographic signature over the identity and the public key. Through this signature, the issuer testifies the subject's ownership of the public key and binds the public key to the identity. The signature of the issuer is verified using the issuer's public key, which is usually presented via another certificate of identical format. This creates a chain of trust in which each certificate is verified by the public key in another certificate, up to the root of trust, which is usually a self-signed certificate whose signature is verified by the public key on the same certificate.

The *Public Key Infrastructure (PKI)* consists of entities that play different roles in this chain of trust. The server (and optionally client) is the leaf node of the chain, and the entities issuing certificates are called *Certificate Authorities (CA)*. The root of this chain is thus referred to as *root CA*, while other issuers between the leaf and the root are referred to

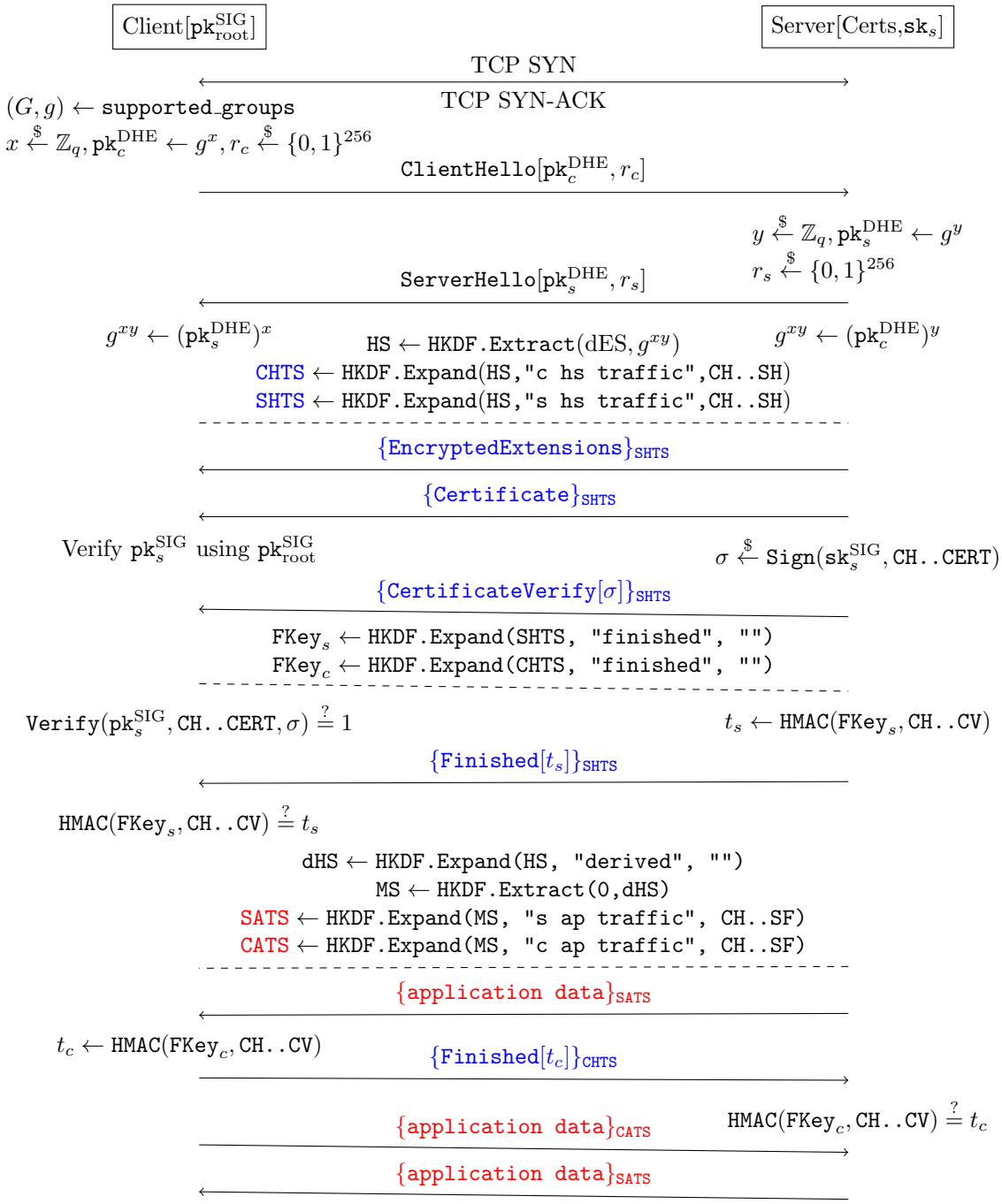

Figure 2.1: TLS 1.3 handshake with unilateral authentication

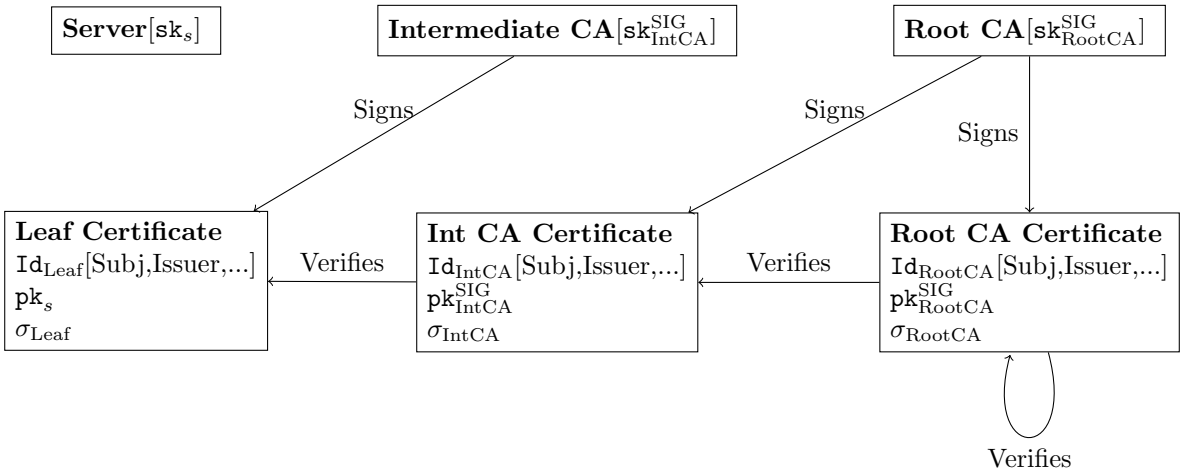

Figure 2.2: Certificate chain and PKI

as intermediate CAs. A selection of trusted root CA certificates are typically pre-installed in client devices by the vendors (e.g. Google, Mozilla, Apple, Microsoft, etc.).

2.1.2 Key schedule and HKDF

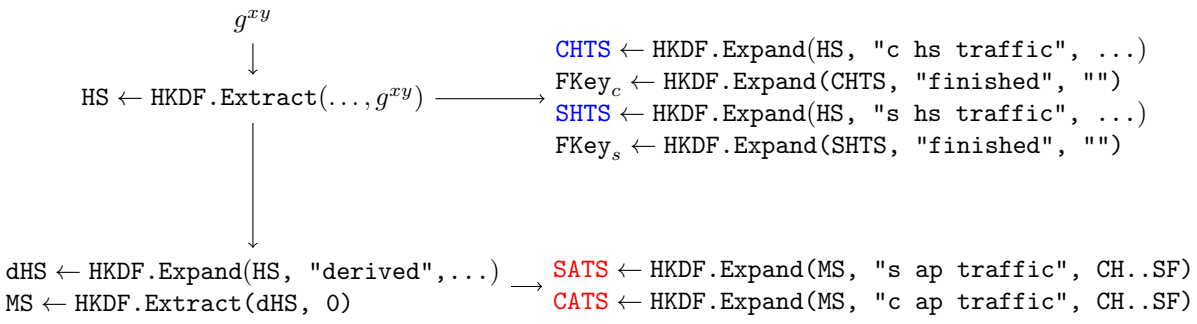

Figure 2.3: HKDF in TLS 1.3 key schedule

TLS 1.3 uses the HMAC-based Key Derivation Function (HKDF) [65] to derive cryptographic keys from input keying material (Figure 2.3). Using an extract-and-expand KDF is necessary because the n-bit shared secret obtained from public-key cryptography (e.g. g^{xy} in a Diffie-Hellman key exchange) is not a uniformly random n-bit string [64]. Instead,

it typically contains fewer bits of computational entropy. [1]

In TLS 1.3, the function `HKDF.Extract` serves as an entropy extractor that concentrates the available randomness in the input keying material. Another critical function of HKDF in TLS 1.3 is to derive computationally independent keys via `HKDF.Expand`. As illustrated in Figure 2.1, all handshake messages following `ServerHello`, as well as all application data, are encrypted using different symmetric keys. Specifically, handshake messages are protected using a pair of handshake traffic keys—client handshake traffic secret (CHTS) and server handshake traffic secret (SHTS)—while application data is encrypted under a separate pair of application traffic keys—client application traffic secret (CATS) and server application traffic secret (SATS). Additionally, there is a unique symmetric key for each party to produce the HMAC tag used in their respective `Finished` messages. According to the HKDF model described in [66, 64], each of these keys—CATS, SATS, CHTS, SHTS, the client finished key, and the server finished key—is computationally independent from all others.

2.1.3 Security analysis

Formal security analysis of TLS 1.3 follows from a long line of work [9, 38, 33, 34, 39] modeling the security of multi-stage authenticated key exchange protocols. Here we provide a high-level description of the model used in [35], although we are glossing over an enormous amount of nuance, and we refer readers to the original paper for rigorous definitions and details.

The primary security goal of TLS 1.3 is *key indistinguishability*, which means that an adversary cannot distinguish secret keys in "fresh" sessions from random keys after interacting with many honest parties across many sessions. By extending the security game across multiple connections with multiple parties, MultiStage security captures auxiliary security goals like forward secrecy and replay attack/downgrade attack resilience. The adversary is a probabilistic polynomial-time Turing machine and can perform a number of malicious actions through various oracles across multiple sessions, including:

- Fooling any honest party into starting a new session with any other honest party (NewSession oracle)
- Sending any message to any party (Send oracle)

[1] As stated in [64], an adversary with knowledge of g^x and g^y can deterministically compute g^{xy}, implying that the statistical entropy of g^{xy} given g^x and g^y is zero.

- Reveal a chosen session key (Reveal oracle), although a session is no longer considered "fresh" if a session key has been revealed
- Compromise the long-term secret of any chosen party (Corrupt oracle)

There are a few constraints on the adversary to prevent trivial wins. For example, the adversary cannot trivially compromise the shared secret from a Diffie-Hellman key exchange. TLS is simply not designed to be secure under these attacks.

At the beginning of the security game, a challenger decides with a coin toss $b \xleftarrow{\$} \{0,1\}$ whether the adversary is in a "NULL" experiment ($b = 0$) or not ($b = 1$). During the game, the adversary is given one more Test oracle that can be queried with any session key identifier. If the session key identifier is invalid (or some other conditions preventing trivial wins are not fulfilled), then the oracle will reject the query. Otherwise, the oracle will return random key or the actual session key depending on the type of experiment. The game ends when the adversary outputs a guess $\hat{b} \in \{0,1\}$ at the type of experiment and wins if the guess is correct.

Under this model, [35] proved that TLS 1.3 is MultiStage secure if the underlying hash function is collision resistant, the signature is existentially unforgeable (EUF-CMA), the Diffie-Hellman key exchange is secure (PRF-ODH), and the KDF is a secure PRF.

Theorem 2.1.1 (Multi-Stage security of `TLS1.3-full-1RTT`)**.** *For any efficient adversary $\mathcal{A}$ against the Multi-Stage security there exist efficient algorithms $\mathcal{B}_1, \ldots, \mathcal{B}_7$ such that*

$$\mathbf{Adv}^{\mathit{Multi\text{-}stage}}_{\mathit{TLS1.3\text{-}full\text{-}1RTT}}(\mathcal{A}) \leq 2n_s \left(\begin{array}{l} \mathbf{Adv}^{\mathit{COLL}}_{H}(\mathcal{B}_1) + n_u \cdot \mathbf{Adv}^{\mathit{EUF\text{-}CMA}}_{\mathit{SIG}}(\mathcal{B}_2) \\ + n_s \left(\begin{array}{l} \mathbf{Adv}^{\mathit{dual\text{-}snPRF\text{-}ODH}}_{\mathit{HKDS.Extract},\mathit{ECDHE}}(\mathcal{B}_3) + \mathbf{Adv}^{\mathit{PRF}}_{\mathit{HKDF.Expand}}(\mathcal{B}_4) \\ + 2 \cdot \mathbf{Adv}^{\mathit{PRF}}_{\mathit{HKDF.Expand}}(\mathcal{B}_5) + \mathbf{Adv}^{\mathit{PRF}}_{\mathit{HKDF.Extract}}(\mathcal{B}_6) \\ + \mathbf{Adv}^{\mathit{PRF}}_{\mathit{HKDF.Expand}}(\mathcal{B}_7) \end{array} \right) \end{array} \right)$$

where n_s is the maximum number of sessions and n_u is the maximum number of users.

2.2 KEMTLS

KEMTLS [88], first proposed by Schwabe, Stebila, and Wiggers in 2020, is a variation on TLS 1.3 that uses key encapsulation mechanism (KEM) instead of digital signature

for peer authentication. KEMTLS is motivatd by the observation that post-quantum signature candidates generally have larger public key and signature than post-quantum KEM candidates' public key and ciphertext, so by using KEMs instead of signatures, one can obtain significant savings in communication sizes.

Section 2.2.1 will introduce the KEMTLS handshake workflow. Section 2.2.2 will review the security properties of KEMTLS and compare them with TLS 1.3.

2.2.1 KEMTLS handshake

Figure 2.4 illustrates a KEMTLS handshake with unilateral authentication. Like TLS 1.3's handshake (Figure 2.1), KEMTLS handshake can be cleanly divided into a key exchange phases where the two parties establish an ephemeral shared secret, and an authentication phase where the server proves its identity to the client. However, KEMTLS' authentication flow diverges significantly from TLS 1.3's authentication flow, and there are subtle downstream differences. We will discuss them in detail in this section.

Ephemeral key exchange. Like TLS 1.3, a KEMTLS handshake begins with establishing a TCP stream and client/server exchanging `ClientHello` and `ServerHello` in this order. Client first picks a KEM scheme $\texttt{KEM}_e$ for ephemeral key exchange, then generates the ephemeral keypair $(\texttt{pk}_e, \texttt{sk}_e) \xleftarrow{\$} \texttt{KEM}_e\texttt{.KeyGen}()$. The ephemeral public key $\texttt{pk}_e$ is sent to the server in `ClientHello`, which also includes a 32-byte client nonce $r_c \in \{0,1\}^{256}$, client's supported cipher suites, and other parameters not crucial to the workflow of KEMTLS. If server accepts the parameters in `ClientHello`, then it encapsulates a random secret $(\texttt{ct}_e, \texttt{ss}_e) \xleftarrow{\$} \texttt{KEM}_e\texttt{.Encap}(\texttt{pk}_e)$. The ciphertext $\texttt{ct}_e$ is included in `ServerHello` sent to the client, alongside server's random nonce $r_s \in \{0,1\}^{256}$ and other parameters. Finally, after receiving `ServerHello`, client recovers the ephemeral shared secret by decapsulating the ciphertext $\texttt{ss}_e \leftarrow \texttt{KEM}_e\texttt{.Decap}(\texttt{sk}_e, \texttt{ct}_e)$.

At this stage, client and server have established an ephemeral shared secret $\texttt{ss}_e$ and will update their key schedule synchronously. Specifically, each extracts a handshake secret (HS) from $\texttt{ss}_e$ using `HKDF.Extract`, then derive client handshake traffic secret (CHTS), server handshake traffic secret (SHTS), and derived handshake secret (dHS) using `HKDF.Expand` under different labels. CHTS and SHTS will be used to encrypt subsequent handshake messages from the client and the server respectively. dHS on the other hand serves as the secret state of the key schedule and will participate in subsequent key schedule update. Figure 2.4 used color-coding and subscripts to clarify which handshake message is encrypted and under which secret.

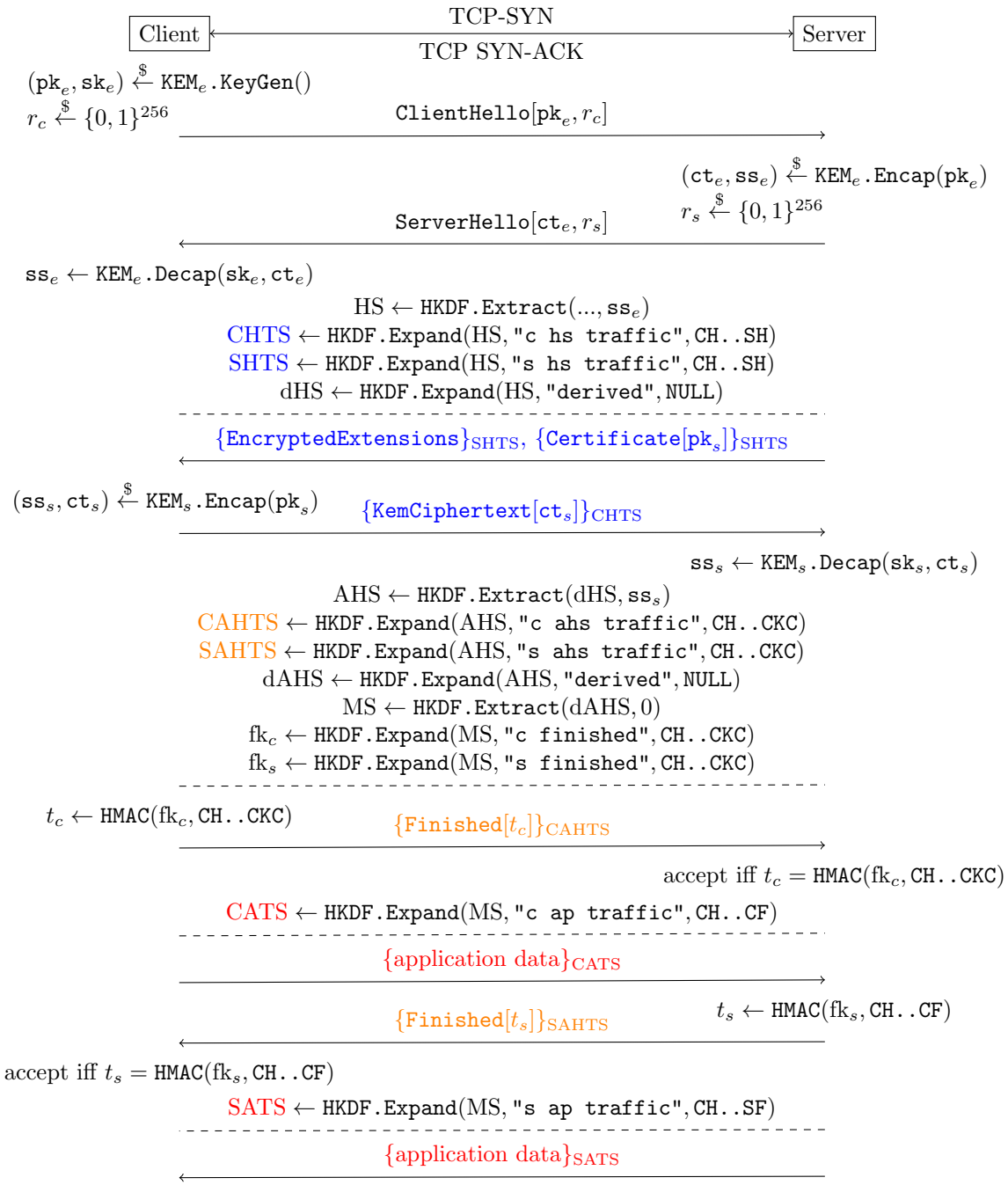

Figure 2.4: KEMTLS handshake with unilateral authentication

(Implicit) Server authentication. Like TLS 1.3, the authentication phase begins with server sending `Certificate`, but the workflow diverges afterwards. With KEMTLS, server's long-term public key $\texttt{pk}_s$ is a KEM encapsulation key. To prove server's possession of the corresponding decapsulation key, client uses server's long-term public key to encapsulate a secret $(\texttt{ct}_s, \texttt{ss}_s) \xleftarrow{\$} \texttt{KEM}_s.\texttt{Encap}(\texttt{pk}_s)$, then sends the ciphertext in a handshake message $\texttt{KemCiphertext}[\texttt{ct}_s]$. After receiving $\texttt{KemCiphertext}[\texttt{ct}_s]$, server recovers the authentication secret using its long-term secret key $\texttt{ss}_s \leftarrow \texttt{KEM}_s.\texttt{Decap}(\texttt{sk}_s, \texttt{ct}_s)$.

After this round of messages, the two parties have obtained another shared secret $\texttt{ss}_s$ and will perform a second update to the key schedule. First, the (unauthenticated) handshake secret (HS) will be upgraded to an authenticated handshake secret (AHS) by extracting randomness from dHS and $\texttt{ss}_s$. From AHS, the handshake traffic secrets will be upgraded to authenticated handshake traffic secrets (CAHTS, SAHTS). The key schedule's secret state is updated from AHS, then used to produce the Master Secret (MS). Last but not least, the `Finished` keys fk_c, fk_s are derived from MS.

Key confirmation/explicit authentication. The final round of handshake message includes client and server exchanging `Finished`. Like TLS 1.3, the `Finished` message contains an HMAC tag over the handshake transcript up to the point when the message is sent. Also like TLS 1.3, each party can derive the appropriate application traffic keys (CATS, SATS) and start sending application data right after sending its own `Finished`. On the other hand, unlike TLS 1.3, the order of "who first sends `Finished` and application" is flipped: where in TLS 1.3, server sends `Finished` before client, in KEMTLS, client sends `Finished` before server. This means that from client's perspective, TLS 1.3 and KEMTLS require the same one round trip (1-RTT) of handshake messages before client can start sending application data. From server's perspective, the number of round trips required before sending application data increased, but as [88] pointed out, in most TLS applications, client sends the first round of application data, so the increase of round trips for server is likely irrelevant to the latency of the application. The flipped order of `Finished` also has security implications, since client will be sending its first round of application data before having received server's `Finished`. We will discuss them in details in Section 2.2.2.

2.2.2 KEMTLS security

KEMTLS has identical security goals to TLS 1.3. Analysis of KEMTLS' security thus follows the same framework as the formal analysis of TLS 1.3's security described in Section

2.1.3. A "pen-and-paper" proof of KEMTLS security properties is first presented in [88]. Later, a computer-verified symbolic analysis of KEMTLS was conducted using Tamarin [27, 74]. At a high level, KEMTLS achieves confidentiality, integrity, authenticity, forward secrecy, and resistance to replays if the client/server nonces are sufficiently large, the ephemeral key exchange KEM is IND-1CCA secure, the authentication KEM is IND-CCA secure, the chosen hash function is collision resistant, the HKDF is a secure PRF, and the AEAD is IND-CCA secure. These properties are formally stated by Theorem 2.2.1:

Theorem 2.2.1 (Multi-stage security of KEMTLS [88])**.** *Let n be the number of sessions and U be the number of parties. The advantage of any probabilistic polynomial-time adversary $\mathcal{A}$ in breaking the multi-stage security of KEMTLS is bounded by:*

$$\mathtt{Adv}(\mathcal{A}) \leq \frac{n^2}{2^{|r_c|}} + \epsilon_H^{COL} + 6n\Bigg(n(\epsilon_{KEM_e}^{IND\text{-}1CCA} + \epsilon_{HKDF.Ext}^{PRF} + 2\epsilon_{HKDF.Ext}^{dual\text{-}PRF} + 3\epsilon_{HKDF.Exp}^{PRF} + \epsilon_{HMAC}^{EUF\text{-}CMA}) + U(\epsilon_{KEM_s}^{IND\text{-}CCA} + 2\epsilon_{HKDF.Ext}^{dual\text{-}PRF} + 2\epsilon_{HKDF.Exp}^{PRF} + \epsilon_{HMAC}^{EUF\text{-}CMA})\Bigg)$$

A note on implicit authentication. In TLS 1.3, client will not receive server's application data nor send its own application data until after it has received and validated server's `Finished`, which means that all application data exchanged are explicitly authenticated. In KEMTLS, however, client can start sending application data after sending its `Finished` but before receiving server's `Finished`. The application data sent before receiving server's `Finished` are encrypted under client's application traffic key, which contains entropy from the authentication secret $\mathtt{ss}_s$. This means that, if $\mathtt{KEM}_s$ is IND-CCA secure, then no party without the long-term seret key $\mathtt{sk}_s$ can correctly obtain the AEAD secret key. In other words, the early application data is **implicitly authenticated**. While this provides less security than explicitly authenticated application data, to this day there is no known attack exploiting the implicit authentication. In addition, explicit authentication can be achieved with just one more round trip, at which time KEMTLS becomes as secure as TLS 1.3 using comparable cryptographic primitives.

Chapter 3

Faster ephemeral key exchange with IND-1CCA KEM

INDistinguishability under (adaptive) **C**hosen **C**iphertext **A**ttacks (IND-CCA) is the desired level of security for a key encapsulation mechanism. Intuitively, IND-CCA security requires that the shared secret from a random encapsulation cannot be distinguished from random bit string of equal length with non-negligible probability, even when the adversary has access to a decapsulation oracle. Theoretically speaking, the number of decapsulation queries is bounded by the requirement that the adversary is "efficient": that is, the adversary, classical or quantum, can only have polynomial time complexity. In practice, NIST's PQC standardization project evaluates a candidate KEM's CCA security under the upper bound of 2^{64} decapsulation queries[75].

However, within the context of an TLS 1.3 (and KEMTLS) key exchange, IND-CCA security is overkill. This is because TLS 1.3 is designed with forward secrecy, which requires that the keypair $(\texttt{pk}_e, \texttt{sk}_e) \xleftarrow{\$} \texttt{KEM}_e\texttt{.KeyGen}()$ exchanged in `ClientHello` and `ServerHello` be freshly generated at every session. Consequently, in a correct TLS 1.3 implementation, each private key will only be used to decapsulate at most one ciphertext. This naturally defines an alternative security definition called IND-1CCA, which is identical to IND-CCA except that the decapsulation oracle will answer at most one query. A similar notion of "security under one query" for Diffie-Hellman-style key exchange protocol called `snPRF-ODH` is defined in [35] and used in the security reduction of TLS 1.3 handshake protocol.

On the other hand, using IND-1CCA security brings meaningful performance improvements. A popular strategy for building an IND-CCA secure KEM is to start with an IND-CPA secure public key encryption scheme (PKE), then apply some variation of the

Fujisaki-Okamoto transformation [40, 41, 54]. The key techniques for achieving chosen-ciphertext security in are *de-randomization* and *re-encryption*. While there is some contention on whether *de-randomiation* hurts security [12] (which indirectly hurts performance because the same security level may require larger set of parameters), there is no doubt that *re-encryption* hurts performance. The performance penalty of re-encryption is especially pronounced with many lattice-based cryptosystems whose CPA-secure encryption sub-routine is much slower than the CPA-secure decryption sub-routine. As we will show in the following section, IND-1CCA constructions can preserve randomization of the underlying PKE while not requiring re-encryption.

In Section 3.1 we will present an original IND-1CCA construction that transforms a CPA-secure PKE into an IND-1CCA secure KEM. Section 3.4 then surveys and discusses other constructions.

3.1 The encrypt-then-MAC transformation

Let $\mathcal{B}^*$ denote the set of finite bit strings. Let $\mathcal{K}_{\mathtt{KEM}}$ denote the set of all possible shared secrets. Let $(\mathtt{KeyGen}_{\mathtt{PKE}}, \mathtt{Enc}_{\mathtt{PKE}}, \mathtt{Dec}_{\mathtt{PKE}})$ be a PKE defined over message space $\mathcal{M}_{\mathtt{PKE}}$ and ciphertext space $\mathcal{C}_{\mathtt{PKE}}$. Let $\mathtt{MAC} : \mathcal{K}_{\mathtt{MAC}} \times \mathcal{B}^* \to \mathcal{T}$ be a MAC over key space $\mathcal{K}_{\mathtt{MAC}}$ and tag space $\mathcal{T}$. Let $G : \mathcal{B}^* \to \mathcal{K}_{\mathtt{MAC}}, H : \mathcal{B}^* \to \mathcal{K}_{\mathtt{KEM}}$ be hash functions. The encrypt-then-MAC transformation $\mathtt{ETM}[\mathtt{PKE}, \mathtt{MAC}, G, H]$ constructs a KEM $(\mathtt{KeyGen}_{\mathtt{ETM}}, \mathtt{Encap}_{\mathtt{ETM}}, \mathtt{Decap}_{\mathtt{ETM}})$ (Figure 3.1).

We chose to construct $\mathtt{KEM}_{\mathtt{ETM}}$ using implicit rejection $K \leftarrow H(s, c)$: on invalid ciphertexts, the decapsulation routine returns a fake shared secret that depends on the ciphertext and some secret values, though choosing to use explicit rejection should not impact the security of the KEM. In addition, because the underlying PKE can be randomized, the shared secret $K \leftarrow H(m, c)$ must depend on both the plaintext and the ciphertext. According to [30, 54], if the input PKE is *rigid* (i.e. $m = \mathtt{Dec}(\mathtt{sk}, c)$ if and only if $c = \mathtt{Enc}(\mathtt{pk}, m)$, such as with RSA), then the shared secret may be derived from the plaintext alone $K \leftarrow H(m)$.

The CCA security of $\mathtt{KEM}_{\mathtt{ETM}}$ can be intuitively argued through an adversary's inability to learn additional information from the decapsulation oracle. For an adversary A to produce a valid tag for some unauthenticated ciphertext c', it must either know the correct symmetric key or produce a forgery. Under the Random Oracle Model (ROM), A cannot know the symmetric key without knowing its pre-image under the hash function G, so A must either produced c' honestly, or have broken the one-wayness of the underlying PKE. This means that the decapsulation oracle will not leak information on decryption that the adversary does not already know. We formalize the security in Theorem 3.1.1

$\mathtt{KeyGen}_{\mathtt{ETM}}()$

1: $(\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}_{\mathtt{PKE}}()$
2: $s \xleftarrow{\$} \mathcal{M}_{\mathtt{PKE}}$
3: $\mathtt{sk} \leftarrow (\mathtt{sk}, s)$
4: **return** $(\mathtt{pk}, \mathtt{sk})$

$\mathtt{Encap}_{\mathtt{ETM}}(\mathtt{pk})$

1: $m \xleftarrow{\$} \mathcal{M}_{\mathtt{PKE}}$
2: $k \leftarrow G(m)$
3: $c' \xleftarrow{\$} \mathtt{Enc}_{\mathtt{PKE}}(\mathtt{pk}, m)$
4: $t \leftarrow \mathtt{MAC}(k, c')$
5: $c \leftarrow (c', t)$
6: $K \leftarrow H(m, c)$
7: **return** (c, K)

$\mathtt{Decap}_{\mathtt{ETM}}(\mathtt{sk}, c)$

Require: $c = (c', t)$
Require: $\mathtt{sk} = (\mathtt{sk}', s)$
1: $\hat{m} \leftarrow \mathtt{Dec}_{\mathtt{PKE}}(\mathtt{sk}', c')$
2: $\hat{k} \leftarrow G(\hat{m})$
3: **if** $\mathtt{MAC}(\hat{k}, c') = t$ **then**
4: $\quad K \leftarrow H(\hat{m}, c)$
5: **else**
6: $\quad K \leftarrow H(s, c)$
7: **end if**
8: **return** K

Figure 3.1: The encrypt-then-MAC KEM routines

Theorem 3.1.1. *For every IND-qCCA adversary A against $\mathtt{KEM}_{\mathtt{ETM}}$ making q decapsulation queries, there exists a OW-PCA adversary B against the underlying PKE making at least q decapsulation queries, and an existential forgery adversary C against the underlying MAC such that:*

$$\mathbf{Adv}^{\mathit{IND\text{-}CCA}}_{\mathit{KEM}_{\mathit{ETM}}}(A) \leq q \cdot \mathbf{Adv}_{\mathit{MAC}}(C) + 2 \cdot \mathbf{Adv}^{\mathit{OW\text{-}PCA}}_{\mathit{PKE}}(B)$$

From here, we can reduce OW-PCA security to OW-CPA security using a guessing argument: a CPA adversary can simulate a PCA oracle simply by randomly sampling from $\{0, 1\}$ as response to every plaintext-checking query. Thus, the probability that all guesses are correct and PCA adversary retains its advantage is at most 2^{-q}. In other words:

Lemma 3.1.2. *For every efficient q-query OW-PCA adversary A against some PKE, there exists some OW-CPA adversary B such that*

$$\mathbf{Adv}^{\mathit{OW\text{-}PCA}}_{\mathit{PKE}}(A) \leq 2^q \cdot \mathbf{Adv}^{\mathit{OW\text{-}CPA}}_{\mathit{PKE}}(B)$$

3.1.1 Proof of Theorem 3.1.1

We will prove Theorem 3.1.1 using a sequence of game. A summary of the the sequence of games can be found in Figure 3.2 and 3.3. From a high level we made three incremental modifications to the IND-CCA game for $\mathtt{KEM}_{\mathtt{ETM}}$:

1. Replace the true decapsulation oracle with a simulated decapsulation oracle. The simulated decapsulation oracle does not directly decrypt the queried ciphertext. Instead it searches through the hash oracle and looks for matching queries using the plaintext-checking oracle. The true decapsulation oracle and the simulated decapsulation oracle disagree if and only if the adversary queries with a ciphertext that contains a forged MAC tag, so the adversary cannot distinguish the two games more than it can perform existential forgery against the underlying MAC.

2. Replace the pseudorandom MAC key $k^* \leftarrow G(m^*)$ with a uniformly random MAC key $k^* \xleftarrow{\$} \mathcal{K}_{\texttt{MAC}}$. Under ROM, the adversary cannot distinguish this game from the previous one unless it queries the hash oracle with m^*, in which case a second adversary with access to a plaintext-checking oracle can win the OW-PCA game against the underlying PKE.

3. Replace the pseudorandom shared secret $K_0 \leftarrow H(m^*, c)$ with a truly random shared secret $K_0 \xleftarrow{\$} \mathcal{K}_{\texttt{KEM}}$. Similar to the point above, the adversary cannot distinguish the change more than it can break the one-wayness of the underlying PKE. Furthermore, since both K_0 and K_1 are uniformly random, no adversary can have any advantage.

A OW-PCA adversary can then simulate the modified IND-CCA game for the KEM adversary, and the advantage of the OW-PCA adversary is associated with the probability of certain behaviors of the KEM adversary.

Proof. Game 0 is the standard KEM IND-CCA game. The decapsulation oracle $\mathcal{O}^{\texttt{Decap}}$ executes the decapsulation routine using the challenge keypair and return the results faithfully. The queries made to the hash oracles $\mathcal{O}^G, \mathcal{O}^H$ are recorded to their respective tapes $\mathcal{L}^G, \mathcal{L}^H$.

Game 1 is identical to game 0 except that the true decapsulation oracle $\mathcal{O}^{\texttt{Decap}}$ is replaced with a simulated oracle $\mathcal{O}_1^{\texttt{Decap}}$. Instead of directly decrypting c' as in the decapsulation routine, the simulated oracle searches through the tape $\mathcal{L}^G$ to find a matching query $(\tilde{m}, \tilde{k})$ such that $\tilde{m}$ is the decryption of c'. The simulated oracle then uses $\tilde{k}$ to validate the tag t against c'.

If the simulated oracle accepts the queried ciphertext as valid, then there is a matching query that also validates the tag, which means that the queried ciphertext is honestly generated. Therefore, the true oracle must also accept the queried ciphertext. On the other hand, if the true oracle rejects the queried ciphertext, then the tag is simply invalid under the MAC key $k = G(\texttt{Dec}(\texttt{sk}', c'))$. Therefore, there could not have been a matching

IND-CCA game for $\mathtt{KEM}_{\mathtt{ETM}}$

1: $(\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}_{\mathtt{ETM}}()$
2: $m^* \xleftarrow{\$} \mathcal{M}$
3: $c' \xleftarrow{\$} \mathtt{Enc}_{\mathtt{PKE}}(\mathtt{pk}, m^*)$
4: $k^* \leftarrow G(m^*)$ ▷ Game 0-1
5: $k^* \xleftarrow{\$} \mathcal{K}_{\mathtt{MAC}}$ ▷ Game 2-3
6: $t \leftarrow \mathtt{MAC}(k^*, c')$
7: $c^* \leftarrow (c', t)$
8: $K_0 \leftarrow H(m^*, c^*)$ ▷ Game 0-2
9: $K_0 \xleftarrow{\$} \mathcal{K}_{\mathtt{KEM}}$ ▷ Game 3
10: $K_1 \xleftarrow{\$} \mathcal{K}_{\mathtt{KEM}}$
11: $b \xleftarrow{\$} \{0, 1\}$
12: $\hat{b} \leftarrow A^{\mathcal{O}^{\mathtt{Decap}}}(\mathtt{pk}, c^*, K_b)$ ▷ Game 0
13: $\hat{b} \leftarrow A^{\mathcal{O}_1^{\mathtt{Decap}}}(\mathtt{pk}, c^*, K_b)$ ▷ Game 1-3
14: **return** $[\![\hat{b} = b]\!]$

Hash oracle $\mathcal{O}^G(m)$

1: **if** $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \tilde{m} = m$ **then**
2: **return** $\tilde{k}$
3: **end if**
4: $k \xleftarrow{\$} \mathcal{K}_{\mathtt{MAC}}$
5: $\mathcal{L}^G \leftarrow \mathcal{L}^G \cup \{(m, k)\}$
6: **return** k

Decap oracle $\mathcal{O}^{\mathtt{Decap}}(c)$

1: $(c', t) \leftarrow c$
2: $\hat{m} = \mathtt{Dec}_{\mathtt{PKE}}(\mathtt{sk}', c')$
3: $\hat{k} \leftarrow G(\hat{m})$
4: **if** $\mathtt{MAC}(\hat{k}, c') = t$ **then**
5: $K \leftarrow H(\hat{m}, c)$
6: **else**
7: $K \leftarrow H(z, c)$
8: **end if**
9: **return** K

$\mathcal{O}_1^{\mathtt{Decap}}(c)$

1: $(c', t) \leftarrow c$
2: **if** $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \tilde{m} = \mathtt{Dec}_{\mathtt{PKE}}(\mathtt{sk}', c') \wedge \mathtt{MAC}(\tilde{k}, c') = t$ **then**
3: $K \leftarrow H(\tilde{m}, c)$
4: **else**
5: $K \leftarrow H(z, c)$
6: **end if**
7: **return** K

$\mathcal{O}^H(m, c)$

1: **if** $\exists(\tilde{m}, \tilde{c}, \tilde{K}) \in \mathcal{L}^H : \tilde{m} = m \wedge \tilde{c} = c$ **then**
2: **return** $\tilde{K}$
3: **end if**
4: $K \xleftarrow{\$} \mathcal{K}_{\mathtt{KEM}}$
5: $\mathcal{L}^H \leftarrow \mathcal{L}^H \cup \{(m, c, K)\}$
6: **return** K

Figure 3.2: Sequence of games in the proof of Theorem 3.1.1

$B(\mathtt{pk}, c'^*)$

1: $z \xleftarrow{\$} \mathcal{M}$
2: $k \xleftarrow{\$} \mathcal{K}_{\mathtt{MAC}}$
3: $t \leftarrow \mathtt{MAC}(k, c'^*)$
4: $c^* \leftarrow (c'^*, t)$
5: $K \xleftarrow{\$} \mathcal{K}_{\mathtt{KEM}}$
6: $\hat{b} \leftarrow A^{\mathcal{O}_B^{\mathtt{Decap}}, \mathcal{O}_B^G, \mathcal{O}_B^H}(\mathtt{pk}, c^*, K)$
7: **if** $\mathtt{ABORT}(m)$ **then**
8: **return** m
9: **end if**

$\mathcal{O}_B^H(m, c)$

if $\mathcal{O}^{\mathtt{PCO}}(m, c'^*) = 1$ **then**
$\mathtt{ABORT}(m)$
end if
if $\exists(\tilde{m}, \tilde{c}, \tilde{K}) \in \mathcal{L}^H : \tilde{m} = m \wedge \tilde{c} = c$ **then**
return $\tilde{K}$
end if
$K \xleftarrow{\$} \mathcal{K}_{\mathtt{KEM}}$
$\mathcal{L}^H \leftarrow \mathcal{L}^H \cup \{(m, c, K)\}$
return K

$\mathcal{O}_B^{\mathtt{Decap}}(c)$

1: $(c', t) \leftarrow c$
2: **if** $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \mathcal{O}^{\mathtt{PCO}}(\tilde{m}, c') = 1 \wedge \mathtt{MAC}(\tilde{k}, c') = t$ **then**
3: $K \leftarrow H(\tilde{m}, c)$
4: **else**
5: $K \leftarrow H(z, c)$
6: **end if**
7: **return** K

$\mathcal{O}_B^G(m)$

1: **if** $\mathcal{O}^{\mathtt{PCO}}(m, c'^*) = 1$ **then**
2: $\mathtt{ABORT}(m)$
3: **end if**
4: **if** $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \tilde{m} = m$ **then**
5: **return** $\tilde{k}$
6: **end if**
7: $k \xleftarrow{\$} \mathcal{K}_{\mathtt{MAC}}$
8: $\mathcal{L}^G \leftarrow \mathcal{L}^G \cup \{(m, k)\}$
9: **return** k

Figure 3.3: OW-PCA adversary B simulates game 3 for IND-CCA adversary A in the proof for Theorem 3.1.1

query that also validates the tag, and the simulated oracle must also rejects the queried ciphertext.

This means that from the adversary A's perspective, game 1 and game 0 differ only when the true oracle accepts while the simulated oracle rejects, which means that t is a valid tag for c' under $k = G(\mathtt{Dec}(\mathtt{sk}', c'))$, but k has never been queried. Under the random oracle model, such k is a uniformly random sample of $\mathcal{K}_{\mathtt{MAC}}$ that the adversary does not know, so for A to produce a valid tag is to produce a forgery against the $\mathtt{MAC}$ under an unknown and uniformly random key. Therefore, we can bound the probability that the true decapsulation oracle disagrees with the simulated oracle by the probability that some MAC adversary produces a forgery:

$$P\left[\mathcal{O}^{\mathtt{Decap}}(c) \neq \mathcal{O}_1^{\mathtt{Decap}}(c)\right] \leq \mathtt{Adv}_{\mathtt{MAC}}(C).$$

Across all q decapsulation queries, the probability that at least one query is a forgery is thus at most $q \cdot P\left[\mathcal{O}^{\mathtt{Decap}}(c) \neq \mathcal{O}_1^{\mathtt{Decap}}(c)\right]$. By the difference lemma:

$$|\mathtt{Adv}_{G_0}(A) - \mathtt{Adv}_{G_1}(A)| \leq q \cdot \mathtt{Adv}_{\mathtt{MAC}}(C).$$

Game 2 is identical to game 1, except that the challenger samples a uniformly random MAC key $k^* \xleftarrow{\$} \mathcal{K}_{\mathtt{MAC}}$ instead of deriving it from m^*. From A's perspective the two games are indistinguishable, unless A queries G with the value of m^*. Denote the probability that A queries G with m^* by $P[\mathtt{QUERY\ G}]$, then:

$$|\mathtt{Adv}_{G_1}(A) - \mathtt{Adv}_{G_2}(A)| \leq P\left[\mathtt{QUERY\ G}\right].$$

Game 3 is identical to game 2, except that the challenger samples a uniformly random shared secret $K_0 \xleftarrow{\$} \mathcal{K}_{\mathtt{KEM}}$ instead of deriving it from m^* and t. From A's perspective the two games are indistinguishable, unless A queries H with $(m^*, \cdot)$. Denote the probability that A queries H with $(m^*, \cdot)$ by $P[\mathtt{QUERY\ H}]$, then:

$$|\mathtt{Adv}_{G_2}(A) - \mathtt{Adv}_{G_3}(A)| \leq P\left[\mathtt{QUERY\ H}\right].$$

Since in game 3, both K_0 and K_1 are uniformly random and independent of all other variables, no adversary can have any advantage: $\mathtt{Adv}_{G_3}(A) = 0$.

We will bound $P[\mathtt{QUERY\ G}]$ and $P[\mathtt{QUERY\ H}]$ by constructing a OW-PCA adversary B against the underlying PKE that uses A as a sub-routine. B's behaviors are summarized in Figure 3.3.

B simulates game 3 for A: upon receiving the public key $\texttt{pk}$ and challenge encryption c'^*, B samples random MAC key and session key to produce the challenge encapsulation, then feeds it to A. When simulating the decapsulation oracle, B uses the plaintext-checking oracle to look for matching queries in $\mathcal{L}^G$. When simulating the hash oracles, B uses the plaintext-checking oracle to detect when $m^* = \texttt{Dec}(\texttt{sk}', c'^\star)$ has been queried. When m^* is queried, B terminates A and returns m^* to win the OW-PCA game. In other words:

$$P\left[\texttt{QUERY G}\right] \leq \texttt{Adv}^{\texttt{OW-PCA}}_{\texttt{PKE}}(B),$$
$$P\left[\texttt{QUERY H}\right] \leq \texttt{Adv}^{\texttt{OW-PCA}}_{\texttt{PKE}}(B).$$

Combining all equations above produce the desired security bound. □

3.2 Practical instantiation with ML-KEM

$\texttt{KEM}^{\not\perp}_m.\texttt{KeyGen}()$

1: $(\texttt{pk}, \texttt{sk}') \xleftarrow{\$} \texttt{PKE.KeyGen}()$
2: $z \xleftarrow{\$} \mathcal{M}$
3: $\texttt{sk} \leftarrow (\texttt{sk}', \texttt{pk}, z)$
4: **return** $(\texttt{pk}, \texttt{sk})$

$\texttt{KEM}^{\not\perp}_m.\texttt{Encap}(\texttt{pk})$

1: $m \xleftarrow{\$} \mathcal{M}$
2: $r \leftarrow G(m)$
3: $c \leftarrow \texttt{PKE.Enc}(\texttt{pk}, m, r)$
4: $K \leftarrow H(m)$
5: **return** (c, K)

$\texttt{KEM}^{\not\perp}_m.\texttt{Decap}(\texttt{sk}, c)$

1: $\hat{m} \leftarrow \texttt{PKE.Dec}(\texttt{sk}', c)$
2: $\hat{r} \leftarrow G(\hat{m})$
3: $\hat{c} \leftarrow \texttt{PKE.Enc}(\texttt{pk}, \hat{m}, \hat{r})$
4: **if** $\hat{c} = c$ **then**
5: $\quad K \leftarrow H(\hat{m})$
6: **else**
7: $\quad K \leftarrow H(z, c)$
8: **end if**
9: **return** K

Figure 3.4: The $\texttt{KEM}^{\not\perp}_m$ variant of FO transformation[54] is used in ML-KEM

ML-KEM is an IND-CCA secure KEM standardized by NIST in FIPS 203 [77]. The CCA security of ML-KEM is achieved in two steps. The first step constructs a PKE (Figure B.1) whose IND-CPA security reduces to the conjectured intractability of the Decisional Module Learning with Error (MLWE) problem. Then the $\texttt{KEM}^{\not\perp}_m$ variant of the FO transformation (Figure 3.4) is applied to convert the IND-CPA secure PKE into an IND-CCA secure KEM.

3.2.1 One-time ML-KEM (OT-ML-KEM)

We apply the encrypt-then-MAC transformation to the K-PKE routines of ML-KEM and call the transformed KEM *one-time ML-KEM (OT-ML-KEM)* (Figure 3.5). The key generation routines are identical between OT-ML-KEM and ML-KEM. The OT-ML-KEM encapsulation and decapsulation routines make additional use a MAC and a KDF.

`OT-ML-KEM.KeyGen()`

1: $z \xleftarrow{\$} \{0,1\}^{256}$
2: $(\mathtt{pk}, \mathtt{sk}') \xleftarrow{\$} \mathtt{K\text{-}PKE.KeyGen}()$
3: $h \leftarrow H(\mathtt{pk})$
4: $\mathtt{sk} \leftarrow (sk' \| \mathtt{pk} \| h \| z)$
5: **return** $(\mathtt{pk}, \mathtt{sk})$

`OT-ML-KEM.Encap(pk)`

1: $m \xleftarrow{\$} \{0,1\}^{256}$
2: $(\overline{K}, k) \leftarrow G(m \| H(\mathtt{pk}))$
3: $c' \xleftarrow{\$} \mathtt{K\text{-}PKE.Enc}(\mathtt{pk}, m)$
4: $t \leftarrow \mathtt{MAC}(k, c')$
5: $K \leftarrow \mathtt{KDF}(\overline{K} \| t)$
6: $c \leftarrow (c', t)$
7: **return** (c, K)

`OT-ML-KEM.Decap(sk, c)`

Require: Secret key $\mathtt{sk} = (\mathtt{sk}' \| \mathtt{pk} \| h \| z)$
Require: Ciphertext $c = (c' \| t)$
1: $(\mathtt{sk}', \mathtt{pk}, h, z) \leftarrow \mathtt{sk}$
2: $(c', t) \leftarrow c$
3: $\hat{m} \leftarrow \mathtt{K\text{-}PKE.Dec}(\mathtt{sk}', c')$
4: $(\overline{K}, \hat{k}) \leftarrow G(\hat{m} \| h)$
5: $\hat{t} \leftarrow \mathtt{MAC}(\hat{k}, c')$
6: **if** $\hat{t} = t$ **then**
7: $\quad K \leftarrow \mathtt{KDF}(\overline{K} \| t)$
8: **else**
9: $\quad K \leftarrow \mathtt{KDF}(z \| c)$
10: **end if**
11: **return** K

Figure 3.5: OT-ML-KEM applies the encrypt-then-MAC transformation to the K-PKE sub-routines of ML-KEM. G is SHA3-512, H is SHA3-256, KDF is Shake256

For performance and practical security, OT-ML-KEM makes two modifications to the encrypt-then-MAC transformation:

- *Hashing public key into* MAC *key and shared secret.* The public key goes into deriving both the MAC key and the shared secret for the same two reasons why Kyber [21] hashes its public key into the pseudorandom coin and the shared secret. First, we want the KEM to be *contributory*, meaning both parties in a two-party key exchange have inputs into the shared secret. Second, we want to enhance the KEM's multi-user security: if the MAC key is derived from the PKE plaintext alone, then an

adversary can pre-compute a large dictionary mapping MAC keys to the pre-image PKE plaintext. Upon receiving some ciphertext, this adversary can search through this key-plaintext dictionary and recover the decryption. Hashing the public key into the MAC key prevents an adversary from using a single lookup table against multiple keypairs.

- *Hashing the* MAC *tag instead of the entire ciphertext.* Because we allowed the PKE to be randomized, the shared secret must have inputs from both the plaintext and the ciphertext. However, we find it unnecessary to hash the entire ciphertext. Instead, if the MAC is modeled as a secure pseudorandom function (PRF), then the MAC tag functions as a (keyed) hash of the ciphertext, so we use the much shorter MAC tag as a substitute.

Compared to ML-KEM, OT-ML-KEM makes the following trade-offs:

- OT-ML-KEM's encapsulation is slightly slower because of the additional steps of computing the MAC tag
- OT-ML-KEM ciphertext size increases by the size of the MAC tag.
- OT-ML-KEM replaces *re-encryption* with computing the MAC tag.
- As the name suggests, each OT-ML-KEM keypair can only be used once.

3.2.2 Choosing a message authentication code

We instantiated OT-ML-KEM with a wide range of MAC designs, including Poly1305 [11], GMAC [72], CMAC [17, 58], and KMAC [50]. All four schemes are parameterized with 256-bit key and 128-bit tag (except for KMAC, see paragraph below) to ensure that MAC keys are at least as hard to guess has the underlying PKE plaintext.

Poly1305 and GMAC are both Carter-Wegman style MAC [109] that compute the tag using finite field arithmetics. Each scheme operates by breaking the message into blocks that can then be parsed into finite field elements. The tag is computed by evaluating a polynomial whose coefficients are the message blocks and whose indeterminate is the secret key. Poly1305 operates in the prime field $\mathrm{GF}(2^{130} - 5)$ whereas GMAC operates in the binary field $\mathrm{GF}(2^{128})$.

CMAC is based on the CBC-MAC with the block cipher instantiated from AES-256. To compute a CMAC tag, the message is first broke into 128-bit blocks with appropriate

padding. Each block is first XORed with the previous block's output, then encrypted under AES using the symmetric key. The final output is XORed with a sub key derived from the symmetric key, before being encrypted for one last time.

KMAC is based on the SHA-3 family of sponge functions [76]. We chose KMAC-256, which uses Shake256 as the underlying extendable output functions. KMAC is the only MAC to support variable key and tag length, though we fixed the key length at 256 bits. On the other hand, we chose the appropriate tag length for the corresponding security level: when instantiated with ML-KEM-512, the tag length is 128 bits; with ML-KEM-768, the tag length is 192 bits; with ML-KEM-1024, the tag length is 256 bits.

3.3 Performance of OT-ML-KEM

We implemented OT-ML-KEM by extending the reference implementation[1] and tested its performance on a desktop processor. All C code is compiled with `GCC 11.4.1` and `OpenSSL 3.0.8`. All binaries are executed on an AWS c7a.medium instance[2] with AMD EPYC 9R14 CPU at 3.7 GHz and 1 GB of RAM in the `us-west-2` region. Source code is available on GitHub[3].

3.3.1 MAC performance

We isolated the performance of OpenSSL's MAC implementations by measuring the CPU cycles consumed to compute a tag on random inputs whose sizes correspond to the ciphertext sizes of ML-KEM at NIST levels 1, 3, and 5. The median CPU cycle counts in 10,000 rounds of testing are summarized in Table 3.1.

Remark. In OpenSSL's implementation, Poly1305 is an one-time secure MAC. OpenSSL does not have a standalone GMAC; instead we used the AEAD `AES-256-GCM` but with all data treated as "associated data". CMAC and KMAC-256 are always many-time secure.

[1] https://github.com/pq-crystals/kyber

[2] We chose the C7 family of EC2 instance because each vCPU in this class of VMs corresponds directly to a physical CPU core instead of a thread on a physical core

[3] https://github.com/xuganyu96/faster-generic-ind-cca-kem-using-encrypt-then-mac/tree/tches2025-volume2

MAC	NIST level 1		NIST level 3		NIST level 5	
	input size	768	input size	1088	input size	1568
Poly1305		909		961		1065
GMAC		3899		4827		4055
CMAC		6291		7588		8735
KMAC-256		6373		9697		11647

Table 3.1: MAC performance (cycles/tick)

3.3.2 KEM routines performance

We ran each KEM routines 10,000 times and measured their CPU cycles. Table 3.2 summarizes the results. Compared to the $\texttt{KEM}^{\not\perp}_{m}$ variant of Fujisaki-Okamoto transformed used in ML-KEM, the encrypt-then-MAC transformation achieves massive CPU cycle count reduction in decapsulation while incurring only a minimal increase of encapsulation cycle count and ciphertext size. Since `K-PKE.Enc` carries significantly more computational complexity than `K-PKE.Dec` or any MAC we chose, the performance advantage of the encrypt-then-MAC transformation over the $\texttt{KEM}^{\not\perp}_{m}$ transformation is dominated by the runtime saving gained from replacing *re-encryption* with MAC.

		KEM	Encap	Decap
NIST level 1		ML-KEM-512	91467	121185
`pk` size	800	OT-ML-KEM-512 + Poly1305	93157	33733
`sk` size	1632	OT-ML-KEM-512 + GMAC	97369	37725
`ct` size	768	OT-ML-KEM-512 + CMAC	99739	40117
`KeyGen` performance (cyc/tick)	75945	OT-ML-KEM-512 + KMAC-256	101009	40741
NIST level 3		ML-KEM-768	136405	186445
`pk` size	1184	OT-ML-KEM-768 + Poly1305	146405	43315
`sk` size	2400	OT-ML-KEM-768 + GMAC	149525	46513
`ct` size	1088	OT-ML-KEM-768 + CMAC	153139	49841
`KeyGen` performance (cyc/tick)	129895	OT-ML-KEM-768 + KMAC-256	155219	52415
NIST level 5		ML-KEM-1024	199185	246245
`pk` size	1568	OT-ML-KEM-1024 + Poly1305	205763	51375
`sk` size	3168	OT-ML-KEM-1024 + GMAC	208805	54573
`ct` size	1568	OT-ML-KEM-1024 + CMAC	213667	59175
`KeyGen` performance (cyc/tick)	194921	OT-ML-KEM-1024 + KMAC-256	216761	62269

Table 3.2: KEM routine performance

3.4 Related works

Optimal Asymmetric Encryption[10] is a generic chosen-ciphertext secure PKE. Under ROM, the chosen-ciphertext security of OAEP reduces to the one-wayness of the input trapdoor permutation. Although there are trapdoor permutations with which OAEP does not achieve full IND-CCA security [93], Fujisaki et al. later proved that OAEP is IND-CCA secure when combined with the RSA trapdoor permutation [42]. However, OEAP's input requirement for a trapdoor permutation is difficult to satisfy, and no other practical instantiation saw widespread adoption to this day.

The *Fujisaki-Okamoto transformation* [40, 41] is another generic IND-CCA secure transformation. The original proposal constructs a hybrid PKE that uses the input PKE to encrypt a symmetric key, then uses the symmetric key to encrypt data. The CCA security of the FO-transformed PKE reduces non-tightly to the one-wayness of the input PKE and the semantic security of the input symmetric cipher. Later works [30, 54, 37, 55] tightened the security reduction, accounted for imperfect correctness, adapted the core techniques to build a KEM, and produced a security reduction under QROM. Thanks to its simplicity and proven security against classical and quantum adversaries, the FO transformation was adopted by many submissions to NIST's PQC standardization project.

The FO transformation is not perfect. Common criticism against FO transformation focuses on *re-encryption*, which is not only slows down decapsulation, but increases risk of side-channel. As demonstrated in [108, 101, 83], *re-encryption* is the entrypoint of many side-channel-assisted plaintext-checking attacks on NIST PQC candidates.

Bounded CCA security is first proposed by Cramer et al.[28], who proposed a generic black-box IND-qCCA PKE construction whose security reduces to the IND-CPA security of the underlying PKE and the existential unforgeability of the underlying one-time signature [67]. Besides being the first of its kind, Cramer's IND-qCCA transformation is also remarkable because the security reduction is proved under the standard model insteasd of the ROM. Unfortunately, this transformation is also computationally ineffiency and thus impractical.

Huguenin-Dumittan and Vaudenay [57] extended this line of work to the post-quantum setting by proposing two IND-qCCA KEM constructions (respectively denoted by T_{CH} and T_H) from CPA-secure PKEs. T_{CH} achieves qCCA security using a "confirmation hash": the KEM ciphertext includes the PKE ciphertext and a hash of the PKE plaintext, and the KEM shared secret is derived from hashing the PKE plaintext. The security reduction is loose by a factor of 2^q where q is the number of decapsulation queries. T_H achieves qCCA security by deriving the shared secret from hashing both the PKE plaintext and

KeyGen
1: $(\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}^{\mathtt{PKE}}()$

Encap(pk)
1: $m \xleftarrow{\$} \mathcal{M}^{\mathtt{PKE}}$
2: $c \xleftarrow{\$} \mathtt{Enc}^{\mathtt{PKE}}(\mathtt{pk}, m)$
3: $K \leftarrow H(m, c)$
4: **return** (c, K)

Decap(sk, c)
1: $\hat{m} \leftarrow \mathtt{Dec}^{\mathtt{PKE}}(\mathtt{sk}, c)$
2: **if** $\hat{m} = \perp$ **then**
3: **return** $\perp$
4: **end if**
5: **return** $H(\hat{m}, c)$

Figure 3.6: T_H transformation

ciphertext (Figure 3.6), so decapsulation queries with modified ciphertext will be answered with independent random values under the rancom oracle model. T_H's construction is simpler than T_{CH} and retains the original ciphertext length, but on the other hand, the security reduction is looser, with a factor of q_H^{2q}, where q_H is the number of hash oracle queries, and q is the number of decapsulation queries. The authors also proved qCCA security of both transformations under quantum random oracle model (QROM).

In the same paper, the two authors also observed that in TLS 1.3, the handshake secret (and thus all subsequent secrets and keys) is derived with input from the handshake transcript, which means that the KEM public key and ciphertext, embedded in `ClientHello` and `ServerHello` respectively, are already hashed into the handshake secret. This is similar to the T_H construction where the shared secret is hashed from PKE plaintext and ciphertext, and indeed the authors proved that TLS 1.3 remains multi-stage secure even if the KEM is only CPA secure. The initial proof presented by the authors had a looseness factor of $O(q^6)$, which was later improved by Zhou et al.[112] to $O(q)$ for randomized PKEs, and $O(1)$ for rigid PKEs.

Chapter 4

Implementing PQ-TLS and KEMTLS on embedded client

To evaluate post-quantum TLS 1.3 and KEMTLS on embedded clients, we implemented post-quantum TLS 1.3 and KEMTLS on top of WolfSSL[1] and using cryptographic primitives from PQClean[2][62]. Source code for this project is available on GithHub[3]. This chapter describes how we implemented post-quantum TLS 1.3 and KEMTLS.

4.1 WolfSSL

WolfSSL is a modern open-source TLS library written in the C programming language. In addition to a complete TLS stack, WolfSSL also includes a robust cryptogrpahy library called WolfCrypt. WolfSSL is optimized for code size, memory footprint, and portability, and WolfCrypt received many industry/government certification including FIPS140-3 from the US federal government and DO-178C DAL-A for avionics. There are other TLS/SSL libraries with support for embedded devices. For example, [24] evaluated post-quantum TLS 1.2 using the open-source mbedTLS library [4]. Unfortunately mbedTLS did not support TLS 1.3 until December 2021. We chose to work with WolfSSL as it is the more popular choice amongst prior works [106, 105, 104, 49], which makes it easier to compare reults.

[1] https://www.wolfssl.com/
[2] https://github.com/PQClean/PQClean/
[3] https://github.com/xuganyu96/embedded-pqtls
[4] https://github.com/Mbed-TLS/mbedtls

4.1.1 Integrating PQC into WolfCrypt

PQClean is a repository of thoroughly tested, high-confidence C implementations of KEM and signature schemes submitted to NIST's PQC standardization project. The "clean" implementations ensure that C programming best practices such as memory safety and integer overflows are enforced and that the correctness and basic security defense such as avoiding branching on secret data are continuously tested. The project also improved developer ergonomics by applying consistent namespacing on exported symbols and a uniform set of APIs across all curated submissions.

PQClean comes with "batteries-included": it contains reusable symmetric primitives (AES, SHA-2, and SHA-3) shared across downstream implementations. The inclusion of shared symmetric primitives made integrating PQClean into WolfCrypt easier, but unfortunately it duplicates existing functionalities from WolfCrypt, which inflates the size of the firmware. In addition, PQClean's symmetric primitives exhibit different performance characteristics from WolfCrypt: for example, WolfCrypt contains an in-house implementation of ML-KEM and ML-DSA, which uses an in-house implementation of Keccak/SHA-3. WolfCrypt's own benchmark showed that on the embedded client, WolfCrypt's SHA-3 to have approx. 10% more throughput than PQClean's `fips202.c`, while WolfCrypt's ML-KEM is roughly twice as fast as the "clean" ML-KEM. In the end, we chose to disable WolfCrypt's ML-KEM/ML-DSA and use the "clean" implementations (using PQClean's symmetric primitives) across the entire project to ensure fair comparison across different schemes. We leave it for future works to incorporate machine-optimized implementations such as from `pqm4`[5] and `ml-kem-native`[6].

While we kept PQClean's symmetric primitives, we had to modify its `randombytes` API for generating random number. PQClean's existing `randombytes` implementation is stateless and asssumes the existence of a filesystem on Linux, BSD, or Win32. While this works on a desktop environment, it does not work with baremetal embedded clients. In addition, most PQC implementations do not expose internal randomness through some `seed` parameter in the public API, so a wholesale replacement of `randombytes` by another RNG implementation will require extensive refactor of the entire project. Fortunately, WolfCrypt provides an abstraction `WC_RNG` with ready-to-go backends on both Linux and baremetal via Pico-SDK. We adapted the stateless `randombytes` API to use the stateful `WC_RNG` by adding a static `WC_RNG *` pointer in PQClean and implementing a function that sets this static pointer to a user-specified instance of `WC_RNG` (Listing 4.1). This also means that all PQClean schemes will source from the same randomness used throughout the TLS

[5] https://github.com/mupq/pqm4

[6] https://github.com/pq-code-package/mlkem-native

handshake.

```
#ifdef HAVE_WC_RNG
#include <wolfssl/wolfcrypt/random.h>
static WC_RNG *g_rng;
void set_wc_rng(WC_RNG *input) {
    g_rng = input;
}
#endif

int randombytes(uint8_t *output, size_t n) {
    void *buf = (void *)output;
    #ifdef HAVE_WC_RNG
    (void)buf;
    return wc_RNG_GenerateBlock(g_rng, output, n);
    #endif
}
```

Listing 4.1: Use `WC_RNG` as backend for PQClean's `randombytes` API

`liboqs`[7] is another repository of high-quality PQC implementations that contain a wider selection of schemes, some of which are automatically integrated from PQClean. We considered using `liboqs`, but found the lack of documentation on baremetal systems (`arm-none-eabi-gcc` toolchain) and the complexity of the project difficult to navigate. PQClean's simple structure allowed us to easily use a single codebase and build for both server (`x86_64` on Linux) and embedded client with no RTOS.

4.1.2 Implementing post-quantum key exchange

Adding new KEM

Incorporating post-quantum KEMs into the key exchange flow of TLS 1.3 is relatively straightforward despite the API difference between a KEM and a Diffie-Hellman key exchange. In fact, WolfSSL already supports ML-KEM for the key exchange phase, which provides a template with which we can integrate HQC and IND-1CCA ML-KEM. RFC 8446 specified that each key exchange group (called `NamedGroup`) is identified with a 16-bit value, which is captured in an enum type `NamedGroup` in WolfSSL. The variants of this enum type are assigned their group point to be directly used in constructing the `key_share`

[7] https://github.com/open-quantum-safe/liboqs

extension the hello messages. The TLS state machine then uses a case-switch block to call the correct lower-level subroutines based on the variant. Figure 4.2 provides an example.

```
static int TLSX_KeyShare_GenPqcKeyClient(WOLFSSL *ssl,
                                         KeyShareEntry* kse) {
    int ret = 0;
    switch (kse->group) {
        case WOLFSSL_ML_KEM_512:
        case WOLFSSL_ML_KEM_768:
        case WOLFSSL_ML_KEM_1024:
            ret = TLSX_KeyShare_GenMlKemKeyClient(ssl, kse);
            break;
        case HQC_128:
        case HQC_192:
        case HQC_256:
            ret = TLSX_KeyShare_GenHqcKeyClient(ssl, kse);
            break;
        case OT_ML_KEM_512:
        case OT_ML_KEM_768:
        case OT_ML_KEM_1024:
            ret = TLSX_KeyShare_GenOtMlKemKeyClient(ssl, kse);
            break;
        default:
            ret = NOT_COMPILED_IN;
            break;
    }
    return ret;
}
```

Listing 4.2: Use `NamedGroup` enum to direct `keygen` calls to the underlying primitive

The explicit case-switch statement stands in contrast with TLS libraries implemented using languages with more sophisticated abstraction such as generics. For example, `rustls`, a TLS library written in Rust, takes advantage of Rust's traits/generics syntax, which enabled user to provide its own cryptographic backend by implementing a `CryptoProvider` struct (Listing 4.3). While the lack of ergonomic generics in the C programming language made much of WolfSSL's codebase verbose and sometimes difficult to read long case-switch or if-else blocks, however, the explicitness also made it easy to reason about the state machine logic. We especially appreciate this flat abstraction when using a visual debugger such as `lldb`.

```rust
pub struct CryptoProvider {
    pub cipher_suites: Vec<SupportedCipherSuite>,
    pub kx_groups: Vec<&'static dyn SupportedKxGroup>,
    pub signature_verification_algorithms: WebPkiSupportedAlgorithms,
    pub secure_random: &'static dyn SecureRandom,
    pub key_provider: &'static dyn KeyProvider,
}
```

Listing 4.3: User can provide custom cryptographic backend to `rustls` through the `CryptoProvider` struct

Memory management

Programming in C brings substantial challenge in memory management, especially on the embedded client, where memory leaks can quickly exhaust the meager amount of available RAM. Following best practices, we allocate memory for cryptographic operations on the stack wherever possible, and on the heap only when necessary, such as when data need to persist across multiple TLS messages. With the ephemeral key exchange, KEM public key and private key must be dynamically allocated because they need to live after the local frame exits. On the other hand, ciphertexts and shared secrets are short-lived and are thus allocated on the stack. Listing 4.4 provides an example of managing memories for KEM keys.

Miscellaneous

One interesting design is a `static const word16 preferredGroup[]`. User can specify the key exchange groups using the `wolfSSL_CTX_set_groups` API, but this user-specified list of groups is copmared with the internal static `preferredGroups` when constructing `ClientHello`. If the two lists do not match, then WolfSSL will send a `ClientHello` with an empty `key_share` extension; per RFC 8446, this indicates that client is requesting group selection from the server, and will increase handshake latency by one round trip. We were unaware of this feature and had spent a lot of effort trying to find out why the client is sending `ClientHello` twice even though client and server both support the newly added KEM.

Last but not least, WolfSSL included ECC+KEM hybrids for key exchange, but the integration with WolfCrypt's ML-KEM is too complex for us to modify in time. We included benchmark results from these hybrid key exchange groups in Section 5.4 but they are not directly comparable with other ML-KEM-only key exchange.

```c
static int TLSX_KeyShare_GenPQCleanMlKemKeyClient(WOLFSSL *ssl,
                                                  KeyShareEntry *kse) {
    int ret = 0;
    PQCleanMlKemKey kem;
    byte *privKey = NULL;
    word32 privKeyLen = 0;
    /* ... fill in privKeyLen, kse->pubKeyLen ... */
    if (ret == 0) {
        privKey = XMALLOC(privKeyLen, ssl->heap,
                          DYNAMIC_TYPE_PRIVATE_KEY);
        if (privKey == NULL)
            ret = MEMORY_ERROR;
    }
    if (ret == 0) {
        kse->pubKey = XMALLOC(kse->pubKeyLen, ssl->heap,
                              DYNAMIC_TYPE_PUBLIC_KEY);
        if (kse->pubKey == NULL)
            ret = MEMORY_ERROR;
    }
    /* ... generate key pair ... */
    if (ret != 0) {
        if (privKey)
            XFREE(privKey, ssl->heap, DYNAMIC_TYPE_PRIVATE_KEY);
        if (kse->pubKey) {
            XFREE(kse->pubKey, ssl->heap, DYNAMIC_TYPE_PUBLIC_KEY);
            kse->pubKey = NULL;
        }
    } else {
        kse->privKey = privKey;
        kse->privKeyLen = privKeyLen;
    }
    return ret;
}
```

Listing 4.4: Managing memory for KEM key

4.1.3 Authenticate with post-quantum signatures

Because we need to distribute related components of a public-key certificate chain to the server and the client separately, we need to generate the certificate and the private key files. X.509 is the standard format for encoding a public-key certificate. It is based on the Abstract Sytnax Notation One (ASN.1) and is usually serialized according to Distinguished Encoding Rule (DER). DER is a binary format and not human-readable. We chose to further encode the DER output into PEM, which is a human-readable format that exclusively uses ASCII characters. Although the PEM-format requires a larger size to store the same certificate/private key because it is a Base64 encoding of the DER data enclosed in header and footer, it is also much easier to work with, as we could trivially load a PEM-formatted certificate into the embedded client's firmware using C macros (Listing 4.5).

```
#define CA_CERT                     \
    "-----BEGIN CERTIFICATE-----"   \
    "..."                           \
    "-----END CERTIFICATE-----"
/* ... setting up WolfSSL ... */
uint8_t ca_certs[] = CA_CERT;
size_t ca_certs_size = sizeof(ca_certs);
wolfSSL_CTX_set_verify(ctx, WOLFSSL_VERIFY_PEER, NULL);
ssl_err = wolfSSL_CTX_load_verify_buffer(ctx, ca_certs, ca_certs_size,
                                         SSL_FILETYPE_PEM);
if (ssl_err != SSL_SUCCESS) {
    /* fail */
}
```

Listing 4.5: PEM-formatted certificate can be loaded without a file system

WolfSSL includes an ASN.1 module that can generate, sign, encode, and parse public-key certificates and private keys (Listing 4.6).

Thanks to existing scaffolding for supporting ML-DSA, expanding WolfSSL's ASN.1 engine to support generating certificates containing ML-KEM/HQC/Falcon/SPHINCS+ public key, and to support signing certificate body with Falcon/SPHINCS+ private keys is straightforward. We particularly appreciate the clever abstraction with which WolfSSL's `MakeCert` and `SignCert` API can support more than a dozen signature scheme/variants while maintaining a compact function signature (Listing 4.7).

Expanding WolfSSL's implementation to parse Falcon/SPHINCS+ public key from certificate and produce/verify Falcon/SPHINCS+ signature in `CertificateVerify` follows similar procedures, so we omit the implementation details. The one exception is the

```c
int ret;
WC_RNG rng;
MlDsaKey key;
Cert cert;
uint8_t der[BUF_SIZE], pem[BUF_SIZE];
int derSz, pemSz;
/* ... initialize signature keypairs and RNG ... */
wc_InitCert(&cert);
cert.sigType = CTC_ML_DSA_LEVEL1;
/* ... fill in subject, issuer, valid dates ... */
derSz = wc_MakeCert_ex(&cert, der, sizeof(der), ML_DSA_LEVEL2_TYPE,
                       &key, &rng);
derSz = wc_SignCert_ex(cert.bodySz, cert.sigType, der, sizeof(der),
                       ML_DSA_LEVEL2_TYPE, &key, &rng);
pemSz = wc_DerToPem(der, derSz, pem, sizeof(pem), CERT_TYPE);
```

Listing 4.6: Generate, sign, then PEM-encode certificate with WolfSSL

```c
int wc_MakeCert_ex(Cert* cert, byte* derBuffer, word32 derSz,
                   int keyType, void* key, WC_RNG* rng) {
    falcon_key*         falconKey = NULL;
    dilithium_key*      dilithiumKey = NULL;
    /* ... other key types ...*/
    switch (keyType) {
        case FALCON_LEVEL1_TYPE:
        case FALCON_LEVEL5_TYPE:
            falconKey = (falcon_key *)key;
            break;
        case ML_DSA_LEVEL2_TYPE:
        case ML_DSA_LEVEL3_TYPE:
        case ML_DSA_LEVEL5_TYPE:
            dilithiumKey = (dilithium_key *)key;
            break;
        /* ... */
    }
    /* ... build cert with key and encode DER to derBuffer ... */
}
```

Listing 4.7: Using enum as hint to cast void pointers to the correct type allows `MakeCert`/`SignCert` to have compact signature despite lack of generics in C

16383-byte fragment size limit of TLS 1.3's record layer. This prevents all SPHINCS+ signatures (except for the "small" variant at NIST level 1) from being sent in a single record. While RFC 8446 allows `CertificateVerify` with signature size up to 65535 bytes to be fragmented across multiple records, unfortunately WolfSSL did not support fragmenting this message. Consequently, we were unable to test server authentication with any of SPHINCS+ fast variants nor any of the small variants at NIST level 3 or above.

4.1.4 Implementing unilaterally authenticated KEMTLS

As illustrated in Figure 2.1 and 2.4, KEMTLS and TLS 1.3 share half of the handshake workflow, and they handle exchanging application data identically. Their similarities allowed us to reuse much of WolfSSL's existing TLS 1.3 state machine, making modifications only where the handshake workflows diverge. This section describe in details these modifications.

Handling KEM certificates

We begin by adding two flags `haveMlKemAuth` and `haveHqcAuth` to the `WOLFSSL` struct. These flags indicate whether server authentication will go through KEMTLS workflow and with which scheme. If client authentication is to be implemented in future works, then we need to account for the possibility that server may wish to be authenticated with KEM while client is authenticated with signatures, hence two sets of flags are needed.

On the server side, if the leaf public key from the loaded certificate chain is a ML-KEM/HQC key, as identified by the OID, then the appropriate flag is set. After server finishes sending `Certificate`, the KEM-auth flags will decide whether the state machine will proceed to send `CertificateVerify` as in signature-based authentication, or to process `KemCiphertext` as in KEM-authentication (Listing 4.8).

On the client side, the KEM-auth flags are set during `Certificate` processing, and they decide whether the client state machine proceeds to process `CertificateVerify` or to send `KemCiphertext` (Listing 4.9).

Constructing and handling `KemCiphertext`

We follow prior implementation [26, 49] for the structure of this message: `KemCiphertext` is a handshake message (borrowing the message type `client_key_exchange` from TLS

```c
int wolfSSL_accept_TLSv13(WOLFSSL *ssl) {
    /*  ... earlier states ... */
    if (!ssl->options.resuming && ssl->options.sendVerify) {
        if ((ssl->error = SendTls13Certificate(ssl)) != 0) {
            WOLFSSL_ERROR(ssl->error);
            return WOLFSSL_FATAL_ERROR;
        }
    }
    if (ssl->options.haveMlKemAuth || ssl->options.haveHqcAuth) {
        ssl->options.acceptState = KEMTLS_ACCEPT_CERT_SENT;
        ssl->error = accept_KEMTLS(ssl);
        if (ssl->error == 0) {
            return WOLFSSL_SUCCESS;
        } else {
            WOLFSSL_ERROR(ssl->error);
            return WOLFSSL_FATAL_ERROR;
        }
    } else {
        ssl->options.acceptState = TLS13_CERT_SENT;
    }
    if (!ssl->options.resuming && ssl->options.sendVerify) {
        if ((ssl->error = SendTls13CertificateVerify(ssl)) != 0) {
            WOLFSSL_ERROR(ssl->error);
            return WOLFSSL_FATAL_ERROR;
        }
    }
    /* ... later states ...*/
}
```

Listing 4.8: `accept_KEMTLS` will take over the handshake workflow after sending `Certificate` if either of `haveMlKemAuth` or `haveHqcAuth` is set

1.2) whose body contains the raw ciphertext bytes (Listing 4.10). In our implementation, each server will load at most one private key, so the content of `KemCiphertext` will be decapsulated or rejected according to that one key. We leave for future works to design a richer structure that annotates the message with some identifier (perhaps an OID) so it can be correctly decapsulated by server loaded with multiple private keys.

`Finished` and application data

`Finished` is the last handshake message in both KEMTLS and TLS 1.3. Syntactically, KEMTLS `Finished` is identical to TLS 1.3 `Finished`: an HMAC tag under the sender's

```c
int wolfSSL_connect_TLSv13(WOLFSSL *ssl) {
    /* ... setup, send ClientHello, process ServerHello, etc. ... */
    while (ssl->options.serverState < SERVER_CERT_COMPLETE) {
        if ((ssl->error = ProcessReply(ssl)) < 0) {
                WOLFSSL_ERROR(ssl->error);
                return WOLFSSL_FATAL_ERROR;
        }
    }
    if (ssl->options.haveMlKemAuth || ssl->options.haveHqcAuth) {
        ssl->error = connect_KEMTLS(ssl);
        if (ssl->error != 0) {
            WOLFSSL_ERROR(ssl->error);
            return WOLFSSL_FATAL_ERROR;
        } else {
            return WOLFSSL_SUCCESS;
        }
    }

    while (ssl->options.serverState < SERVER_FINISHED_COMPLETE) {
        if ((ssl->error = ProcessReply(ssl)) < 0) {
            WOLFSSL_ERROR(ssl->error);
            return WOLFSSL_FATAL_ERROR;
        }
    }
    /* ... cleanup ... */
}
```

Listing 4.9: Advance client's state machine until after `Certificate` is processed, then `connect_KEMTLS` will take over the handshake if either of `haveMlKemUath` or `haveHqcAuth` is set while processing `Certificiate`

```
static int SendKemTlsClientKemCiphertext(WOLFSSL *ssl) {
    /* add both record and handshake headers */
    AddTls13Headers(output, ssl->kemCiphertextSz, client_key_exchange,
                    ssl);
    XMEMCPY(input + HANDSHAKE_HEADER_SZ, ssl->kemCiphertext,
            ssl->kemCiphertextSz);
    inputSz = ssl->kemCiphertextSz + HANDSHAKE_HEADER_SZ;
    sendSz = BuildTls13Message(ssl, output, outputSz, input, inputSz,
                               handshake, hashOutput, 0, 0);
    if (sendSz < 0)
        return BUILD_MSG_ERROR;
    ssl->buffers.outputBuffer.length += sendSz;
    ret = SendBuffered(ssl);
    return ret;
}
```

Listing 4.10: Client constructing `KemCiphertext`

`finished_key` against the running hash of the handshake transcript. Therefore we made no change to the construction or processing of this message. On the other hand, there are several semantic differences that require modification for KEMTLS. First, the pair of `finished_key` is derived with input from both the handshake secret and the authentication secret (Listing 4.11). Second, the order in which client and server's `Finished` is sent is flipped, which requires modifying the corresponding "message order sanity checks" in WolfSSL.

KEMTLS's application data layer is identical to its TLS 1.3 counterpart and requires no changes, but before KEMTLS transitions to exchanging application data, we need to derive the application traffic keys and configure the handshake state machine to indicate a successful handshake. KEMTLS and TLS 1.3 both derive application traffic keys from the Master Secret (MS), so we can reuse WolfSSL's `DeriveTls13Keys` after `deriveKemTlsFinishedSecrets` derives MS from HS and $\texttt{ss}_s$. As for configuring the handshake state machine, we observe that WolfSSL keeps track of the state of a handshake using an increasing sequence of integers (Listing 4.12). We added handshake states unique to KEMTLS while making sure that the value of individual states is consistent with the order in which a KEMTLS handshake goes through these states. To ensure that the KEMTLS handshake correctly produced a consistent pair of application traffic keys, we implemented the client to send a few random bytes after the handshake finishes, and the server to echo back client's application data. The application data do not count toward handshake performance in Section 5.4.

```
static int deriveKemTlsFinishedSecrets(WOLFSSL *ssl,
                                       byte *kemSharedSecret,
                                       word32 kemShareddSecretSz) {
    /* ... after sending Finished, derive HS, dHS, AHS, dAHS, and MS */
    /* client_finished_key <- HKDF_expand(MS, "c finished", NULL) */
    DeriveKeyMsg(ssl, ssl->keys.client_write_MAC_secret, -1,
                 ssl->arrays->masterSecret, clientFinishedKeyLabel,
                 clientFinishedKeyLabelLen, NULL, 0,
                 ssl->specs.mac_algorithm);
    /* server_finished_key <- HKDF_expand(MS, "s finished", NULL) */
    DeriveKeyMsg(ssl, ssl->keys.server_write_MAC_secret, -1,
                 ssl->arrays->masterSecret, serverFinishedKeyLabel,
                 serverFinishedKeyLabelLen, NULL, 0,
                 ssl->specs.mac_algorithm);
    return 0;
}
```

Listing 4.11: Deriving `HS, dHS, AHS, dAHS, MS, finished keys`

4.2 Embedded client networking

TLS requires a reliable transport layer such as TCP/IP. In our implementation, server-side networking is trivially handled by the Linux operating system: we only need to call `wolfSSL_set_fd` to connect the TLS state machine to the TCP stream. Client-side networking is non-trivial due to the lack of an operating system. Instead we had to use WolfSSL's I/O callback API and manually connect the transport layer to the TLS state machine. In this section we briefly describe how we approached implementing the transport layer on the embedded client.

We programmed our embedded client (see Section 5.1) using the Pico-SDK, which includes the `cyw43` driver for wireless connectivity and the `lwip` library for low-level protocol control. `lwip` (Lightweight IP) is an open-source TCP/IP stack developed by Adam Dunkels [36]. It is designed for minimal code size and memory usage, making it particularly well-suited for resource-constrained environments.

Unlike prior work evaluating PQ-TLS/KEMTLS [49], we chose not to use a real-time operating system (RTOS). This decision helped us avoid the added complexity of asynchronous network programming typically required with an RTOS. However, it also meant we could not use `lwip`'s higher-level socket API, which depends on RTOS features such as threading and synchronization. Instead, we used `lwip`'s raw API, which offers fine-grained control over TCP but only in an event-driven, callback-based form. Since WolfSSL's I/O

```
static int ssl_in_handshake(WOLFSSL *ssl, int sending_data) {
    if (ssl->options.handShakeState != HANDSHAKE_DONE)
        return 1;
    if (ssl->options.side == WOLFSSL_SERVER_END) {
        if (IsAtLeastTLSv1_3(ssl->version))
            return ssl->options.acceptState < TLS13_TICKET_SENT;
        return 0;
    }
    if (ssl->options.side == WOLFSSL_CLIENT_END) {
        if (IsAtLeastTLSv1_3(ssl->version))
            return ssl->options.connectState < FINISHED_DONE;
        return 0;
    }
    return 0;
}
```

Listing 4.12: WolfSSL checks the state of using the enum `handShakeState` and `connectState`/`acceptState`. Smaller value in this enum indicates earlier state in the handshake.

callback interface is designed to work with stream-oriented transports, we needed to build a stream-like abstraction on top of `lwip`'s raw API.

`lwip`'s raw API handles data at packet level. When the network interface receives an IP packet, a handler is called to process the packet based on higher level need. To get a stream-like behavior, we implemented a buffered read: when a packet is received, the handler will append the incoming packet onto an internal buffer, then later when the application reads from the stream, a pointer keeps track of the amount of data consumed from the internal buffer. The read buffer is freed when all data have been consumed by the application to prevent memory leak. WolfSSL already buffers its write operations, so we did not implement buffered write at the stream level. Listing 4.13 shows how the buffered read is implemented.

```c
/* handle incoming packet */
static err_t tcp_recv_handler(void *arg, struct tcp_pcb *tpcb,
                              struct pbuf *p, err_t err) {
    tcp_stream_t *stream = (tcp_stream_t *)arg;
    err_t lwip_err = ERR_OK;
    if (stream == NULL) {
        /*... fail ...*/
    } else if (!p) {
        stream->terminated = true; /* peer terminated connection */
    } else if (p) {
        if (stream->rx_pbuf) {
            uint32_t remaining_cap = 0xFFFF - stream->rx_pbuf->tot_len;
            if (remaining_cap < p->tot_len) {
                lwip_err = ERR_WOULDBLOCK;
            } else {
                pbuf_cat(stream->rx_pbuf, p);
            }
        } else {
            stream->rx_pbuf = p;
        }
    }
    return lwip_err;
}

err_t tcp_stream_read(tcp_stream_t *stream, uint8_t *buf, size_t bufcap,
                      size_t *outlen) {
    /* ... */
    if (stream->rx_pbuf != NULL) {
        uint16_t remlen = stream->rx_pbuf->tot_len - stream->rx_offset;
        size_t copylen = MIN(remlen, bufcap);
        *outlen = pbuf_copy_partial(stream->rx_pbuf, buf, copylen,
                                    stream->rx_offset);
        tcp_recved(stream->pcb, *outlen);
        if (stream->rx_pbuf->tot_len == *outlen + stream->rx_offset) {
            pbuf_free(stream->rx_pbuf);
            stream->rx_pbuf = NULL;
            stream->rx_offset = 0;
        } else {
            stream->rx_offset += *outlen;
        }
    } /* ... */
}
```

Listing 4.13: Buffered read using `lwip`'s raw API

Chapter 5

Performance of post-quantum TLS 1.3 and KEMTLS

This section will present and analyze performance benchmarking data. Section 5.1 describes the hardware setup and the software toolchain. Section 5.2 analyzes the communication costs, including the sizes of the various certificate chains. Section 5.3 summarizes the performance of relevant cryptographic primitives on the test hardware. Finally Section 5.4 details the performance of post-quantum TLS 1.3 and KEMTLS.

5.1 Experiment setup

Client The Raspberry Pi Pico 2 W (Figure 5.1) is a microcontroller developed by the Raspberry Pi Foundation and was released in late 2024. At the heart of the Pico 2 W is the RP2350 chip (clocked at 150MHz), which contains two ARM Cortex-M33 cores and two Hazard3 RISC-V cores, although only one architecture can be enabled at a time. The Pico 2 W also has 520KB of SRAM and 4MB of flash storage. For connectivity, the Pico 2 W has a CYW43439 module that supports 2.4 GHz Wifi and Bluetooth 5.4. Firmware is compiled using the GNU ARM toolchain (`arm-none-eabi-gcc` version 8.5.0) and `CMakeBuildType=RelWithDebInfo`.

Server The TLS server runs on a desktop computer with an 8-core AMD Ryzen 7 1700X clocked at 3.4GHz and 12GB of DDR4 RAM. The desktop runs Ubuntu 24.04 LTS.

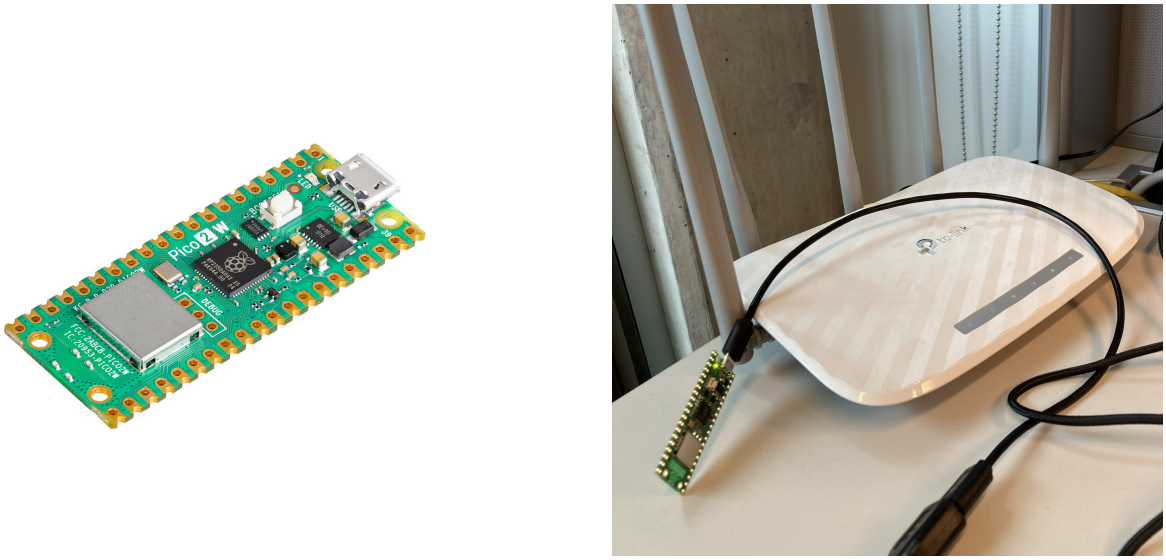

Figure 5.1: Client hardware: Pi Pico 2 W connected via 2.4GHz WiFi

The server binary is compiled using `GCC 13.3.0` and `CMAKE_BUILD_TYPE=RelWithDebInfo`. Turbo boosting was disabled from the BIOS.

Network Our network setup takes advantage of Pico's wireless capabilities. The Pico is connected to a TP-LINK wireless access point via 2.4GHz WiFi (Figure 5.1). The access point is then connected via Ethernet to the internal network of University of Waterloo. The server is directly connected to the same internal network via Ethernet, as well. According to `traceroute`, there is one more hop (an internal nameserver) between the access point and the server. The average network latency is less than 10 milliseconds. We choose to connect to an internal network with Internet access instead of a direct connection because we want to simulate a realistic peer verification policy that checks the expiration dates of peer's certificates. To achieve this, we adapted the example Network Time Protocol (NTP) client from Pico-SDK's repository [1] and synchronized time with `pool.ntp.org`. We also used `lwip`'s DNS API to resolve the server IP address from its hostname. Time synchronization and DNS lookup are both performed once at Pico 2 W's boot time and do not count toward TLS handshake measurements.

[1] https://github.com/raspberrypi/pico-examples/blob/master/pico_w/wifi/ntp_client

5.2 Communication cost

TLS 1.3 recommended ephemeral elliptic curve Diffie-Hellman key agreement (ECDHE) to be the default scheme for key exchange in `ClientHello` and `ServerHello`. Using ECDHE, the client and the server each sends the other an element from the agreed group of points on an elliptic curve, so round-trip communication cost is the sum of two group points. When using a post-quantum KEM, the client sends to the server an encapsulation key, and the server sends back a ciphertext obtained under said encapsulation key. The round-trip communication cost of a KEM key agreemnt is thus the sum of a public key and a ciphertext. Table 5.1 compares the communication costs between ECDHE and various post-quantum KEMs. Note that NIST curves (P-256, P-384, P-521) can be represented using compressed or uncompressed forms, but per RFC 8446 [84], ECDHE in TLS 1.3 always uses the uncompressed form, hence only uncompressed forms are listed.

Table 5.1: Key exchange communication costs (bytes)

Category	Scheme	Public key	Ciphertext	Round trip
ECDHE	P-256	65		130
	P-384	97		194
	P-521	133		266
	X25519	32		64
	X448	56		112
PQC	ML-KEM-512	800	768	1568
	ML-KEM-768	1184	1088	2272
	ML-KEM-1024	1568	1568	3136
	HQC-128	2249	4433	6682
	HQC-192	4522	9029	13551
	HQC-256	7245	14469	21714

Post-quantum digital signature schemes also have larger sizes than elliptic-curve signatures and RSA-signatures. The round trip cost of digital signature is the sum of public key size and signature size. Table 5.2 compares the sizes of public key and signature across RSA, elliptic-curve, and a selection of post-quantum signature schemes.

In practice, the `Certificate` message often contains a certificate chain. In our experiment setup, we generate a server certificate chain with three certificates: a leaf certificate for which the server holds the private key, an intermediate certificate authority who signs server's publickey, and a root certificate authority who signs the intermediate certificate authority's public key. However, root CA's certificate is directly compiled into the client

Table 5.2: Digital signatures communication costs (bytes)

Category	Scheme	Public key	Signature	Total
Classical	RSA-2048 (PKCS#1 v1.5)	256	256	512
	ECDSA P-256	64	64	128
	ECDSA P-384	96	96	192
	ECDSA P-521	132	132	264
	Ed25519	32	64	96
	Ed448	57	114	171
PQC	ML-DSA-44	1312	2420	3732
	ML-DSA-65	1952	3309	5261
	ML-DSA-87	2592	4627	7219
	Falcon-512	897	666	1563
	Falcon-1024	1793	1280	3073
	SPHINCS+-128f	32	17088	17120
	SPHINCS+-192f	48	35664	35712
	SPHINCS+-256f	64	49856	49920
	SPHINCS+-128s	32	7856	7888
	SPHINCS+-192s	48	16224	16272
	SPHINCS+-256s	64	29792	29856

firmware. so server's `Certificate` only contains the leaf certificate and the intermediate CA certificate. Table 5.3 compares the size of root CA and server's certificate chain size across a variety of configurations. Note that these sizes are measured with PEM encoding, whereas the `Certificate` message contains certificates encoded using DER. Since PEM is the base-64 encoding of the raw bytes of DER plus a human-readable header and footer, the sizes presented in Table 5.3 are proportionally larger than what is actually transmitted in a handshake.

5.3 Cryptographic operations performance

We benchmarked the speed of relevant cryptographic operations on the Pico 2 W and the server. On the Pico 2 W, CPU cycle count is read from the `CYCCNT` register, located at `PPB_BASE + M33_DWT_CYCCNT_OFFSET`. On the x86 server, CPU cycle count is read from the `RDTSC` register.

Root	Root cert size	Intermediate	Leaf	Server chain size
		Classical		
RSA-2048	1265	RSA-2048	RSA-2048	2506
ECDSA (P-256)	729	ECDSA (P-256)	ECDSA (P-256)	1429
ECDSA (P-384)	810	ECDSA (P-384)	ECDSA (P-384)	2406
ECDSA (P-521)	912	ECDSA (P-521)	ECDSA (P-521)	2711
Ed25519	639	Ed25519	Ed25519	1893
Ed448	741	Ed448	Ed448	2199
		PQ-TLS		
ML-DSA-44	5596	ML-DSA-44	ML-DSA-44	16763
Falcon-512	2630	Falcon-512	Falcon-512	7874
Falcon-512	2630	Falcon-512	ML-DSA-44	8447
Falcon-512	2634	ML-DSA-44	ML-DSA-44	11404
ML-DSA-65	7668	ML-DSA-65	ML-DSA-44	14446
Falcon-1024	4678	Falcon-1024	Falcon-1024	14017
Falcon-1024	4686	Falcon-1024	ML-DSA-65	14249
Falcon-1024	4686	Falcon-1024	ML-DSA-87	15110
ML-DSA-65	7668	ML-DSA-65	ML-DSA-65	22979
ML-DSA-87	10320	ML-DSA-87	ML-DSA-87	30936
SPHINCS-128f	23710	ML-DSA-44	ML-DSA-44	54734
SPHINCS-128s	11206	ML-DSA-44	ML-DSA-44	29726
SPHINCS-128s	11206	SPHINCS-128s	Falcon-512	34772
SPHINCS-128s	11206	SPHINCS-128s	ML-DSA-44	34772
SPHINCS-128s	11206	SPHINCS-128s	SPHINCS-128s	33594
SPHINCS-192s	22561	SPHINCS-192s	ML-DSA-65	70246
SPHINCS-256s	40956	SPHINCS-256s	Falcon-1024	125187
SPHINCS-256s	40956	SPHINCS-256s	ML-DSA-87	126276
		KEMTLS		
ML-DSA-44	5596	ML-DSA-44	ML-KEM-512	16073
ML-DSA-44	5596	ML-DSA-44	HQC-128	18035
Falcon-512	2634	Falcon-512	ML-KEM-512	7744
ML-DSA-65	7668	ML-DSA-65	ML-KEM-512	13751
ML-DSA-65	7668	ML-DSA-65	ML-KEM-768	21939
ML-DSA-65	7668	ML-DSA-65	HQC-128	15713
Falcon-1024	4682	Falcon-1024	ML-KEM-768	13201
Falcon-1024	4678	Falcon-1024	ML-KEM-1024	13721
ML-DSA-87	10320	ML-DSA-87	ML-KEM-1024	29547
ML-DSA-87	10320	ML-DSA-87	ML-KEM-1024	29547
ML-DSA-87	10320	ML-DSA-87	ML-KEM-768	29027
SPHINCS-128s	11206	SPHINCS-128s	HQC-128	36608
SPHINCS-128s	11206	SPHINCS-128s	ML-KEM-512	34646
SPHINCS-192s	22561	SPHINCS-192s	HQC-192	73728
SPHINCS-192s	22561	SPHINCS-192s	ML-KEM-768	69206
SPHINCS-256s	40956	SPHINCS-256s	HQC-256	132577
SPHINCS-256s	40956	SPHINCS-256s	ML-KEM-1024	124891

Table 5.3: Server certificate chain size (bytes).

Name	Verify
NIST Level 1	
RSA-2048	2.63
Ed25519	2.52
ECDSA (P-256)	21.98
Falcon-512	0.84
ML-DSA-44	3.34
SPHINCS-128-FAST	156.45
SPHINCS-128s	52.69
NIST Level 3	
Ed448	10.93
ECDSA (P-384)	51.56
ML-DSA-65	5.72
SPHINCS-192-FAST	216.40
SPHINCS-192s	75.76
NIST Level 5	
sha521ecdsa	106.35
ML-DSA-87	9.84
Falcon-1024	1.673
SPHINCS-256-FAST	230.20
SPHINCS-256s	113.61

Name	KeyGen	Agree/Encap	Decap
NIST Level 1			
X25519	1.94	1.94	–
SECP256R1	32.97	32.95	–
ML-KEM-512	0.94	1.04	1.22
OT-ML-KEM-512	0.93	1.17	0.44
HQC-128	48.80	98.43	148.21
NIST Level 3			
SECP384R1	79.20	79.17	–
X448	8.76	8.75	–
ML-KEM-768	1.50	1.69	1.94
OT-ML-KEM-768	1.50	1.87	0.62
HQC-192	146.05	299.63	449.69
NIST Level 5			
SECP521R1	168.36	168.33	–
ML-KEM-1024	2.35	2.59	2.91
OT-ML-KEM-1024	2.35	2.86	0.83
HQC-256	272.38	547.03	821.84

Table 5.4: Signature verification and ECDHE/PQ-KEM performance on RP2350 (median million cycles/tick)

Name	KeyGen	Sign	Verify
NIST Level 1			
RSA-2048	526951	11485	165
Ed25519	125	114	354
ECDSA (P-256)	3589	3667	2474
Falcon-512	49129	14716	170
ML-DSA-44	323	1340	352
SPHINCS-128-FAST	8642	202200	12018
SPHINCS-128s	566975	50314	4040
NIST Level 3			
ECDSA (P-384)	9052	9176	6053
Ed448	488	490	1524
ML-DSA-65	589	2239	573
SPHINCS-192-FAST	13136	340445	18110
SPHINCS-192s	814181	923756	5992
NIST Level 5			
ML-DSA-87	890	2354	928
sha521ecdsa	14506	14682	9548
Falcon-1024	128922	32039	347
SPHINCS-256-FAST	33788	684294	17846
SPHINCS-256s	552131	179745	9405

Name	KeyGen	Agree/Encap	Decap
NIST Level 1			
x25519	332	331	–
ECDHE (P-256)	3630	3580	–
ML-KEM-512	126	148	192
OT-ML-KEM-512	128	159	70
HQC-128	7247	14857	23522
NIST Level 3			
x448	1284	1288	–
ECDHE (P-384)	9154	9051	–
ML-KEM-768	209	235	293
OT-ML-KEM-768	211	250	95
HQC-192	22001	44690	68769
NIST Level 5			
ECDHE (P-521)	14473	14515	–
ML-KEM-1024	322	343	418
OT-ML-KEM-1024	328	364	112
HQC-256	40459	81118	127123

Table 5.5: Signature/ECDHE/PQ-KEM performance on Ryzen 1700X (median kilocycles/tick)

5.4 Handshake latency

To analyze and compare handshake latency data, we modified the WolfSSL library to collect timestamps at important steps of the handshake on the client side. The timestamps are collected using the CPU clock, which is not a real-time clock but can precisely measure time interval (within 1-2 microseconds per 150,000 microseconds). The timestamps include:

- `ch_start`: the beginning of the handshake
- `keygen_start`: when client starts generating the ephemeral keypair
- `keygen_done`: when client finishes generating the ephemeral keypair
- `ch_sent`: when client finishes sending `ClientHello`
- `sh_start`: when client start processing `ServerHello`
- `decap_start`: when client start processing server's key share in `ServerHello`. With ECDHE, this means running the `agree` sub-routine; with KEMs, this means running the `decaosulation` sub-routine
- `decap_done`: when client finishes processing server's key share, but before updating key schedule
- `sh_done`: when client finishes processing `ServerHello`, which includes updating key schedule and deriving handshake secret
- `cert_start`: when client starts processing `Certificate`
- `auth_start`: when client starts the asymmetric cryptographic operation for server authentication. In signature-based workflow, this is right before client starts verifying the signature in `CertificateVerify`. In KEM-based workflow, this is right before client starts encapsulating using server's public key.
- `auth_done`: when client finishes the asymmetric cryptographic operation for server authentication
- `hs_done`: when client finishes processing server's `Finished`. Note that in signature-based workflow, this is the earliest moment when client can start sending application data, but in KEM-based workflow, client can start at an earlier moment in the handshake. Due to time constraint we did not get to implement this "early application data", but we speculate that the performance impact will be minimal.

Although communication cost contrinutes significantly to the latency of a handshake in both the key exchange phase and the authentication phase, measuring standalone "certificate transmission time" is not straightforward as it requires high precision time synchronization between the client and the server. Unfortunately, the NTP client can have up to hundreds of milliseconds of bias and is thus rather not helpful. It is also questionable whether standalone transmission time is all that meaningful, because data transmission often happens concurrently with data processing. For example, server transmitting `Certificate` can happen concurrently with client processing `ServerHello`. Therefore, longer transmission time by itself does not necessarily translate to longer handshake latency. Instead, we will only measure the combined duration of data processing and data transmission. Finally, each configuration is repeated for at least 1,000 times.

Key exchange metrics Within the key exchange phase we mainly care about three metrics:

- *Total time*: the duration of the entire key exchange from the client's perspective, which is the interval from `ch_start` to `sh_done`.
- *CPU time*: the time spent constructing `ClientHello` and processing `ServerHello`, equivalently the time during which client's CPU is doing active work. It is computed as `ch_sent - ch_start + sh_done - sh_start`.
- *Crypto time*: the time spent on generating client's key share and processing server's key share. With ECDHE, this means running `keygen` and `agree` each once; with KEM, this means running `keygen` and `decap` each once. Crypto time should be a subset of CPU time, since the client needs to process other components of the two messages and update the key schedule to derive handshake secret. It is computed as `keygen_done - keygen_start + decap_done - decap_start`

Key exchange	Crypto time		CPU time		Total time	
	Median	Std	Median	Std	Median	Std
NIST level 1						
P-256	381534.5	125.85	383791.5	144.38	388032.5	2854.89
X25519	26913.0	5.38	29024.5	42.34	31593.5	5059.76
ML-KEM-512	15831.0	86.41	20030.0	140.38	24741.5	8941.26
OT-ML-KEM-512	10356.0	32.69	14065.0	39.22	20840.0	4206.79
HQC-128	1314945.0	15.04	1322884.0	58.63	1341244.5	5498.32
NIST level 3						
P-384	1069659.0	429.63	1071922.0	434.90	1080339.0	3748.19
X448	118485.0	25.76	120654.0	48.13	123820.0	8489.68
ML-KEM-768	24274.0	119.43	28985.0	217.70	33967.5	5717.48
OT-ML-KEM-768	15262.0	47.17	20071.0	57.84	26195.5	3441.57
HQC-192	3993142.0	20.39	4004725.0	95.34	4040805.0	6251.72
NIST level 5						
P-521	2216709.0	145.19	2218991.5	173.34	2231103.0	2421.84
ML-KEM-1024	36455.0	134.64	41991.0	137.78	51070.5	5987.77
OT-ML-KEM-1024	22302.0	64.37	27879.0	68.68	36102.5	2453.64
HQC-256	7296353.0	25.59	7314304.0	139.59	7380111.0	11151.01

Table 5.6: Key exchange phase duration (μs)

Authentication metrics For the authentication phase we mainly care about two metrics:

- *Crypto time*: the time spent on asymmetric cryptographic operations during the authentication phase. For signature-based authentication, this includes verifying the signature in `Certificate` verify. For KEM-based authentication, this includes encapsulating the authentication secret. Note that in our KEMTLS implementation, the authentication secret is encapsulated eagerly while processing server's `Certificate`, and constructing `KemCiphertext` only involves copying the ciphertext from the internal state to the output buffer, so authentication's crypto time does not involve `KemCiphertext`.
- *Total time*: the duration of the entire authentication phase, which counts from `cert_start` to `hs_done`.

Certificate chain			Crypto time		Total time	
Root	Int	Leaf	Median	Std	Median	Std
NIST Level 1						
RSA-2048	RSA-2048	RSA-2048	17930.0	26.03	80593.0	128.60
Ed25519	Ed25519	Ed25519	16423.0	118.44	75621.0	229.35
ECDSA (P-256)	ECDSA (P-256)	ECDSA (P-256)	128629.0	1239.37	405919.0	1273.67
ML-DSA-44	ML-DSA-44	ML-DSA-44	23248.0	21.35	117080.5	473.17
ML-DSA-44	ML-DSA-44	ML-KEM-512	13328.0	29.75	140961.5	18934.54
Falcon-512	Falcon-512	ML-DSA-44	23177.0	18.14	82447.5	364.27
Falcon-512	Falcon-512	Falcon-512	6654.0	8.60	58831.0	322.49
Falcon-512	Falcon-512	ML-KEM-512	14915.0	33.44	107118.5	17081.24
SPHINCS-128s	SPHINCS-128s	ML-DSA-44	6666.0	6.47	763353.0	412.39
SPHINCS-128s	SPHINCS-128s	Falcon-512	6668.0	7.33	785569.0	737.73
SPHINCS-128s	SPHINCS-128s	SPHINCS-128s	339141.5	7758.01	1130286.0	7723.88
SPHINCS-128s	SPHINCS-128s	ML-KEM-512	14465.0	32.09	841305.0	32294.14
SPHINCS-128s	SPHINCS-128s	HQC-128	663889.0	31.43	1480765.0	8998.96
ML-DSA-65	ML-DSA-65	ML-KEM-512	13608.0	31.50	180912.5	7019.93
NIST Level 3						
Ed448	Ed448	Ed448	75382.5	445.79	266462.0	1806.87
ECDSA (P-384)	ECDSA (P-384)	ECDSA (P-384)	334915.0	2891.93	1027251.5	2899.69
ML-DSA-65	ML-DSA-65	ML-DSA-65	38417.0	20.57	176606.0	743.49
ML-DSA-65	ML-DSA-65	ML-KEM-768	17898.0	31.36	190830.5	9042.40
ML-DSA-87	ML-DSA-87	ML-KEM-768	18430.0	32.95	249737.5	3306.57
Falcon-1024	Falcon-1024	ML-DSA-65	39575.0	30.08	116527.0	242.57
Falcon-1024	Falcon-1024	ML-KEM-768	17939.5	30.90	124804.5	3175.23
SPHINCS-192s	SPHINCS-192s	ML-DSA-65	39606.0	34.21	1168531.0	781.00
SPHINCS-192s	SPHINCS-192s	ML-KEM-768	17086.0	28.74	1162462.5	17567.99
SPHINCS-192s	SPHINCS-192s	HQC-192	2003681.0	30.27	3156085.0	14530.18
NIST Level 5						
ECDSA (P-521)	ECDSA (P-521)	ECDSA (P-521)	686970.0	4300.04	2082331.0	4280.15
ML-DSA-87	ML-DSA-87	ML-DSA-87	65371.0	20.25	274107.5	745.21
ML-DSA-87	ML-DSA-87	ML-KEM-1024	24457.0	31.45	258661.5	3805.98
Falcon-1024	Falcon-1024	ML-DSA-87	67465.0	35.45	152648.0	514.94
Falcon-1024	Falcon-1024	Falcon-1024	12380.0	10.69	82915.0	497.63
Falcon-1024	Falcon-1024	ML-KEM-1024	23645.0	30.02	133261.5	2427.82
SPHINCS-256s	SPHINCS-256s	ML-DSA-87	65518.0	30.69	1711808.5	1232.87
SPHINCS-256s	SPHINCS-256s	Falcon-1024	12728.0	11.59	1617799.0	1399.10
SPHINCS-256s	SPHINCS-256s	ML-KEM-1024	23893.0	31.55	1723219.5	17025.75
SPHINCS-256s	SPHINCS-256s	HQC-256	3653333.0	29.30	5442551.0	11790.08

Table 5.7: Authentication durations (μs)

KEX	Auth			Latency	
	Root	Int	Leaf	Median	Std
Classical					
P-256	RSA-2048	RSA-2048	RSA-2048	486970.0	103761.19
P-256	ECDSA P-256	ECDSA P-256	ECDSA P-256	810259.5	21448.88
X25519	Ed25519	Ed25519	Ed25519	322543.0	94536.37
P-384	ECDSA P-384	ECDSA P-384	ECDSA P-384	2125437.5	6455.31
X448	Ed448	Ed448	Ed448	512519.0	78039.25
P-521	ECDSA P-521	ECDSA P-521	ECDSA P-521	4327127.5	4050.10
PQ-TLS					
ML-KEM-512	ML-DSA-44	ML-DSA-44	ML-DSA-44	164019.5	7742.55
ML-KEM-512	Falcon-512	Falcon-512	ML-DSA-44	128625.0	2595.18
ML-KEM-512	Falcon-512	Falcon-512	Falcon-512	102441.0	843.83
ML-KEM-512	SPHINCS-128s	SPHINCS-128s	ML-DSA-44	832089.0	5538.95
ML-KEM-512	SPHINCS-128s	SPHINCS-128s	Falcon-512	870800.5	30137.05
OT-ML-KEM-512	ML-DSA-44	ML-DSA-44	ML-DSA-44	161103.0	4288.22
HQC-128	SPHINCS-128s	SPHINCS-128s	SPHINCS-128s	2540080.0	128924.41
ML-KEM-768	ML-DSA-65	ML-DSA-65	ML-DSA-65	244529.0	6588.06
ML-KEM-768	Falcon-1024	Falcon-1024	ML-DSA-65	176427.5	6183.20
ML-KEM-768	SPHINCS-192s	SPHINCS-192s	ML-DSA-65	1286830.5	29914.98
OT-ML-KEM-768	ML-DSA-65	ML-DSA-65	ML-DSA-65	239182.0	12654.08
HQC-192	SPHINCS-192s	SPHINCS-192s	ML-DSA-65	5299501.0	304464.20
ML-KEM-1024	ML-DSA-87	ML-DSA-87	ML-DSA-87	366860.5	16685.48
ML-KEM-1024	Falcon-1024	Falcon-1024	Falcon-1024	157197.0	7805.75
ML-KEM-1024	Falcon-1024	Falcon-1024	ML-DSA-87	228236.0	4998.10
ML-KEM-1024	SPHINCS-256s	SPHINCS-256s	ML-DSA-87	1893276.0	39903.86
ML-KEM-1024	SPHINCS-256s	SPHINCS-256s	Falcon-1024	1792750.0	36736.82
OT-ML-KEM-1024	ML-DSA-87	ML-DSA-87	ML-DSA-87	349064.0	19547.31
HQC-256	SPHINCS-256s	SPHINCS-256s	ML-DSA-87	9243921.0	40152.74
KEMTLS					
ML-KEM-512	ML-DSA-44	ML-DSA-44	ML-KEM-512	189151.5	20774.16
ML-KEM-512	ML-DSA-65	ML-DSA-65	ML-KEM-512	241421.0	28136.83
ML-KEM-512	Falcon-512	Falcon-512	ML-KEM-512	157687.0	37145.59
ML-KEM-512	SPHINCS-128s	SPHINCS-128s	ML-KEM-512	925493.0	74438.44
HQC-128	SPHINCS-128s	SPHINCS-128s	HQC-128	2918399.0	223598.25
ML-KEM-768	ML-DSA-65	ML-DSA-65	ML-KEM-768	275167.0	29293.25
ML-KEM-768	ML-DSA-87	ML-DSA-87	ML-KEM-768	326002.0	35235.57
ML-KEM-768	Falcon-1024	Falcon-1024	ML-KEM-768	183946.5	16618.27
ML-KEM-768	SPHINCS-192s	SPHINCS-192s	ML-KEM-768	1277371.5	51458.02
HQC-192	SPHINCS-192s	SPHINCS-192s	HQC-192	7304369.0	243299.18
ML-KEM-1024	ML-DSA-87	ML-DSA-87	ML-KEM-1024	354981.0	35428.57
ML-KEM-1024	Falcon-1024	Falcon-1024	ML-KEM-1024	210244.5	30846.48
ML-KEM-1024	SPHINCS-256s	SPHINCS-256s	ML-KEM-1024	1899104.0	226068.66
HQC-256	SPHINCS-256s	SPHINCS-256s	HQC-256	12987320.5	445509.98

Table 5.8: Handshake latency (μs)

5.4.1 Discussion

Comparing PQ-TLS performance with classical TLS At comparable NIST security level, lattice-based primitives showed competitive if not superior performance compared to RSA and elliptic-curve primitives (Figure 5.2). For ephemeral key exchange, ML-KEM-512 consumed less CPU time and overall time than X25519. ML-KEM-768 consumed approximately equal amount of CPU time as X25519, but consumed more overall time, likely due to the larger public key and ciphertext sizes. We speculate that the competitive performance can be largely attributed to the fact that ML-KEM and ML-DSA have smaller arithmetics: RSA-2048 operates 2048-bit integers, X25519/Ed25519 oeprates with 256-bit integers, while ML-KEM (using a modulus of 3349) and ML-DSA (using a modulus of 8380417) arithmetics all fit within the 32-bit register of the RP-2350.

As expected, SPHINCS+ and HQC performed worse. HQC for key exchange consumed 50-100x more CPU time than X25519/X448 and ML-KEM at comparable security level. On the other hand, signature verification for the small variant of SPHINCS+ consumed approx. 15-20 times the CPU time as ECC and lattice-based scheme.

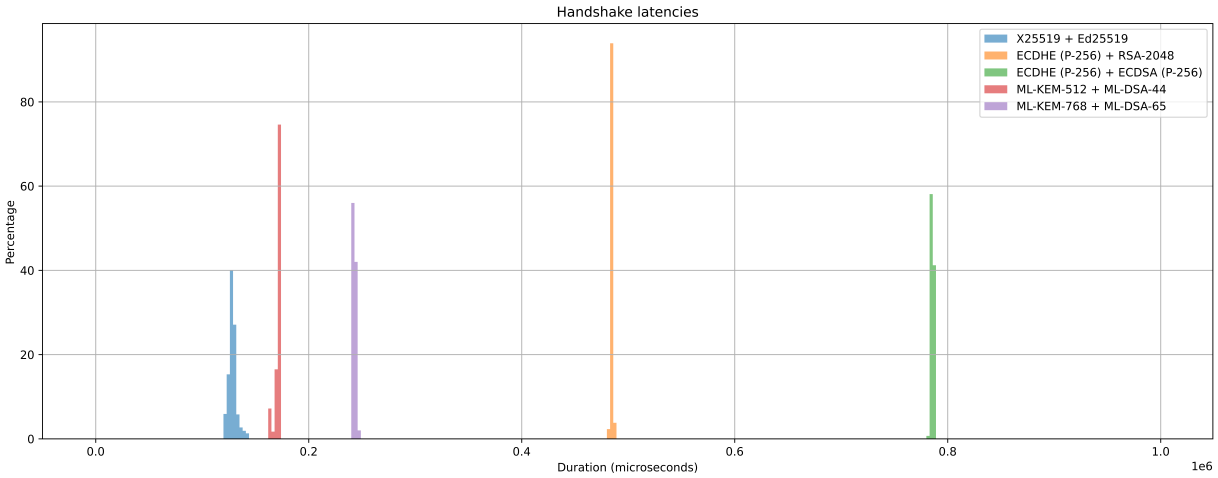

Figure 5.2: Handshake latency: lattice-based vs number-theoretic cipher suite

IND-1CCA KEM We expected meaningful performance improvements from removing *re-encryption* from an FO-protected IND-CCA secure KEM. Prior implementations on SIKE/Kyber/Lightsaber [57] and ML-KEM [111] reported decapsulation speedup ranging from 2x to 6x. Where server and client both have powerful desktop CPUs, cryptographic

operations do not bottleneck the handshake, and the improvements in decapsulation speed only translate to minimal improvements in handshake speed (Table 5.9).

KEM	**Signature**	**Connections per second**	
		FO transform	T_H transform
ML-KEM-512	Dilithium-2	3416.02	3476.17
ML-KEM-768	Dilithium-3	2710.85	2803.24

Table 5.9: IND-1CCA ML-KEM integration in TLS 1.3 [111]

On the other hand, with a constrained TLS client, client-side cryptographic operations pose a more severe bottleneck, so the speed improvements are more pronounced. Another consideration unique to an embedded client is energy consumption: as reported in [103, 87], the choice of cryptographic primitives can greatly affect energy consumption, and thus the lifetime of a battery-operated embedded device.

Figure 5.3 compares the distribution of key exchange latency and the handshake latecy between ML-KEM (IND-CCA secure) and one-time ML-KEM (IND-1CCA secure).

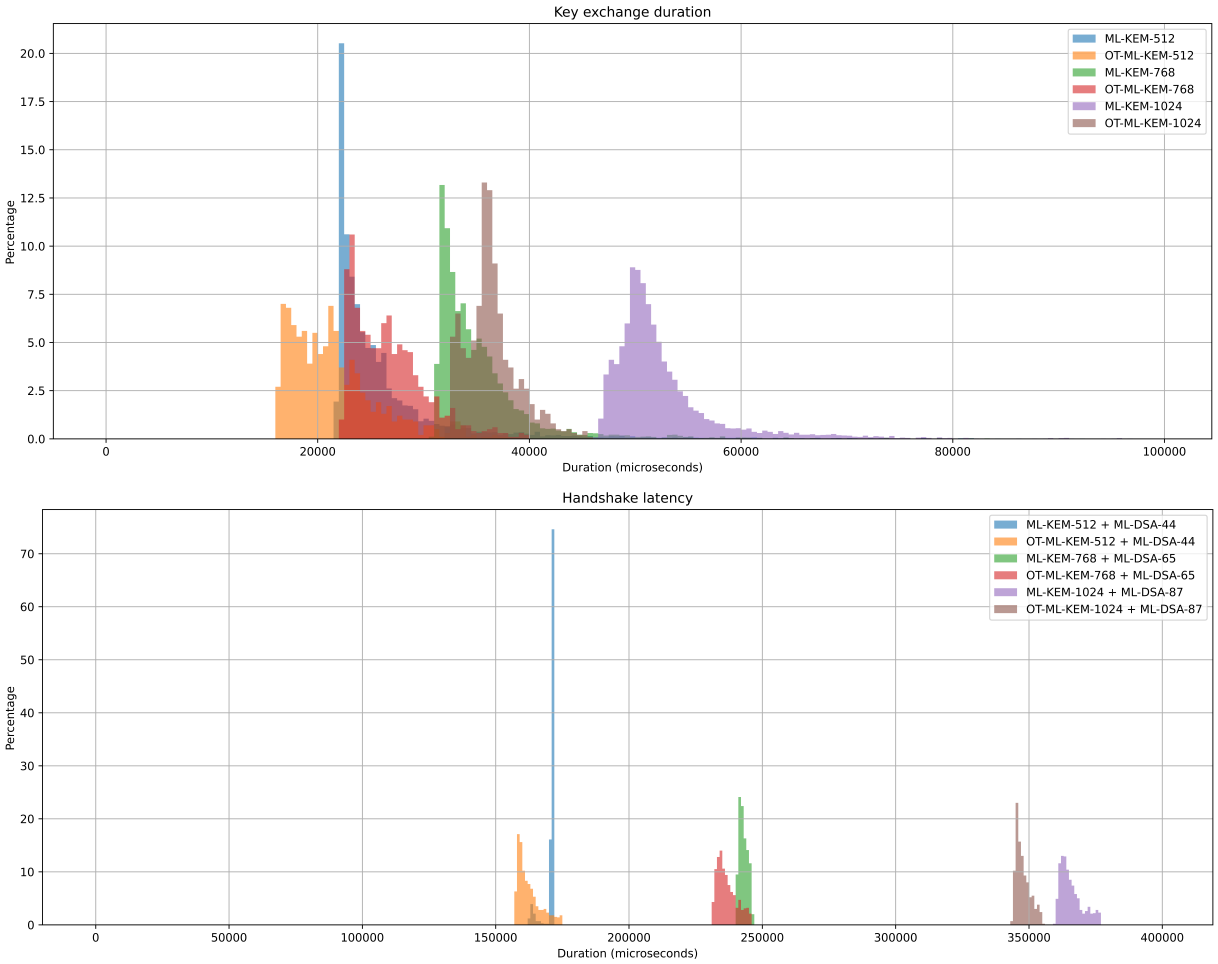

Figure 5.3: Comparing key exchange latency between CCA and 1CCA ML-KEM

PQ-TLS vs KEMTLS In our setup, KEMTLS and PQ-TLS showed comparable performance with respect to handshake latency. As shown in Figure 5.4, encapsulating an authentication secret is not any faster than verifying the signature in `CertificateVerify`. The communication cost advantage of KEMTLS is also partially mitigated by the fact that in a multi-core processor such as the RP2350 in our client device, receiving `Certificate` can happen concurrently with processing `ServerHello`. We speculate that our client can receive `Certificate` much faster than it can decapsulate the ciphertext in `ServerHello`, hence a shorter certificate chain ended up offering no meaningful performance advantage. Other implementations [26, 49] simulated congested or narrow-band networks, which

demonstrated more sizeable performance advantage to KEMTLS.

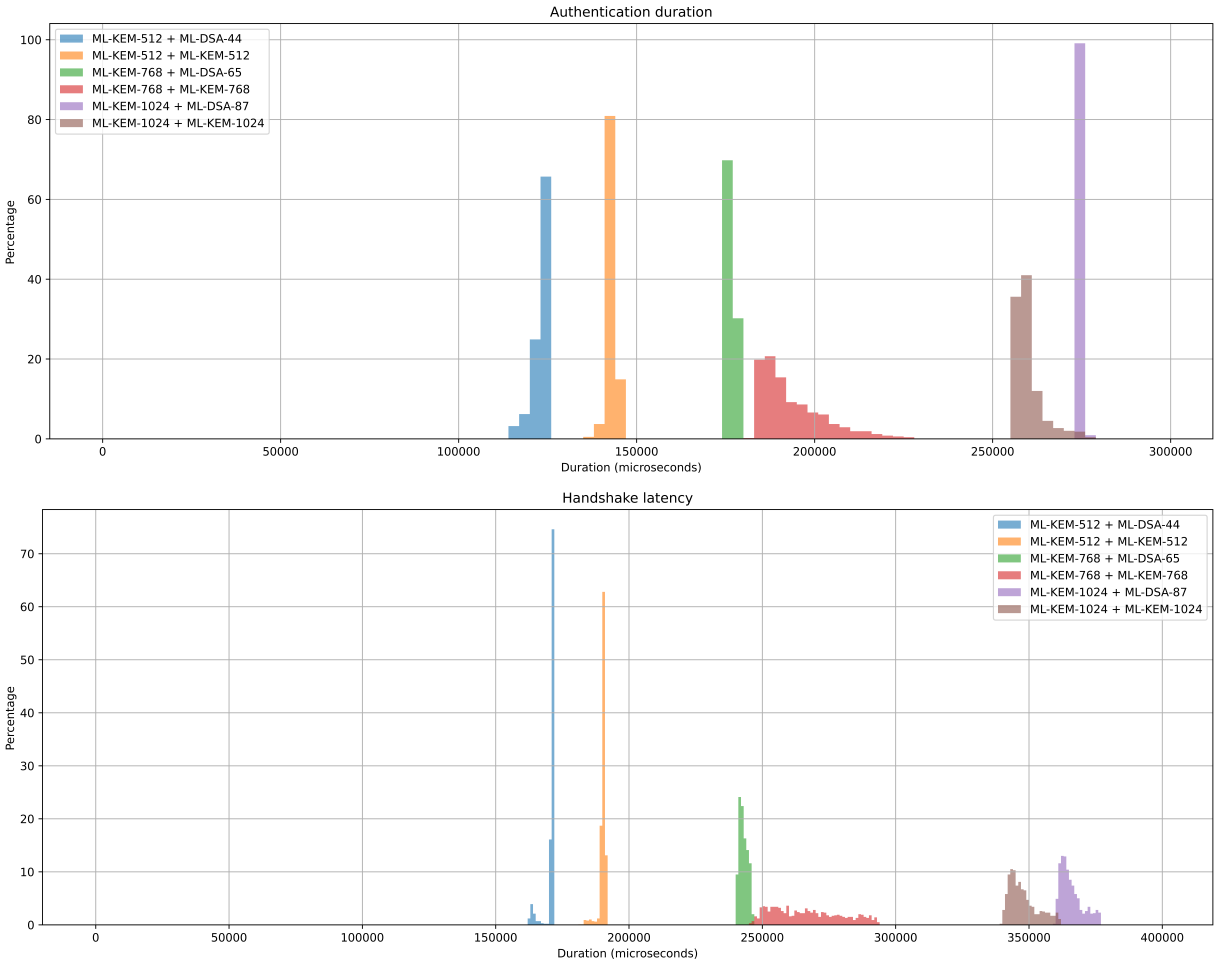

Figure 5.4: Comparing KEMTLS with PQ-TLS

Practical recommendations ML-DSA and Falcon are both lattice-based digital signature schemes selected by NIST for standardization (ML-DSA was standardized in August 2024; Falcon was selected but still in the process of being standardized). However, there are notable differences between their designs and performance characteristics that make one suitable than the other in certain scenarios. Falcon has smaller signature size (752 vs 2420 bytes at NIST level 1, 1462 vs 4627 at level 5) and smaller public key size (897 vs 1312 at level 1, 1793 vs 2592 at level 5). According to eBACS [16], on a Cortex-A72 (Raspberry

Pi 4B)[2], Falcon has significantly faster signing and verification than ML-DSA. Because ML-DSA signs using the "Fiat-Shamir with abort" [68] strategy, signing time can have large variances depending on the input randomness, while Falcon's signing routine does not have this issue. On the other hand, Falcon's signing routine uses a discrete Gaussian sampler, which requires constant-time floating point arithmetics with at least 53-bits of precision. The float-point arithmetics requirement is not only difficult to fulfill, such as on legacy CPUs and/or embedded devices, but also difficult to implement without risk of introducing side-channel vulnerabilities.

While each signature scheme can make a valid certificate chain, given the performance and security characteristics mentioned above, we recommend using Falcon only for offline signatures (signatures are produced ahead of the handshake) while using ML-DSA for online signature (signatures are produced during handshake). For example, root and intermediate CAs can issue certificates using the Falcon, and server signs `CertificateVerify` with ML-DSA. Figure 5.5 compares the performance across various Falcon/ML-DSA certificate chain configurations.

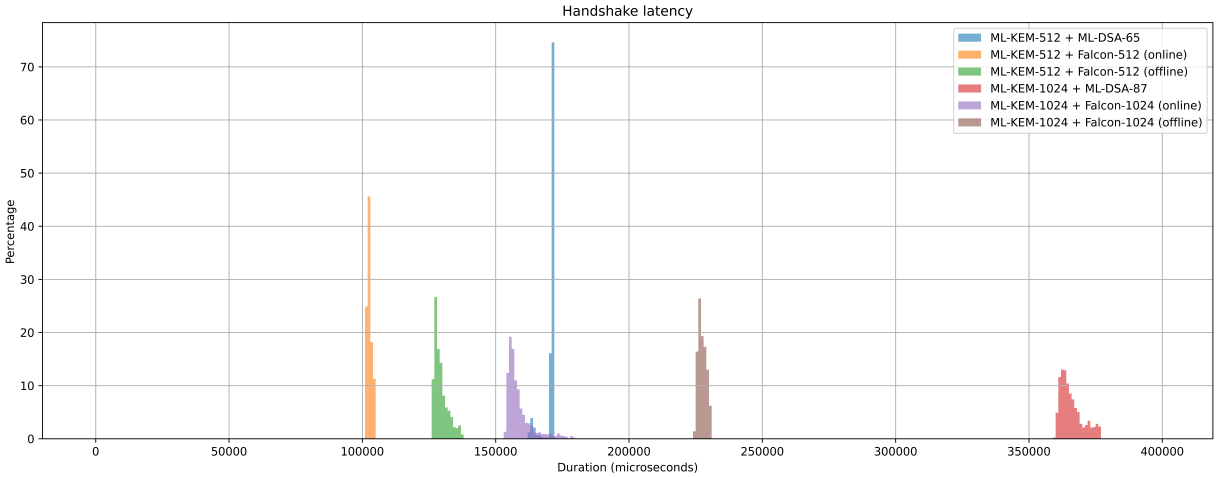

Figure 5.5: Comparing ML-DSA/Falcon mixture chains

[2] https://bench.cr.yp.to/results-sign/aarch64-pi4b.html

Chapter 6

Conclusion and future works

In this thesis, we proposed an original IND-1CCA KEM transformation and applied it to post-quantum TLS handshake on an embedded device, achieving meaningful performance improvements.

In Chapter 2, we reviewed the handshake protocol of TLS 1.3 and KEMTLS. Both protocols used ephemeral key exchange to encrypt subsequent traffic, but they took different approaches to authenticate peers: TLS 1.3 authenticates peer using digital signatures, and KEMTLS authenticates peer using CCA-secure KEMs. Both protocols allow clients to send application data after 1-RTT, but the different authentication approaches brought subtle security implications: in signature-based authentication, 1-RTT application data is explicitly authenticated, but in KEM-based authentication, 1-RTT application data is only implicitly authenticated. KEMTLS can still achieve explicitly authenticated application data, but only at the cost of one more round trip.

In Chapter 3 we introduced "bounded CCA security (IND-1CCA)", then proposed a generic IND-1CCA KEM transformation that builds from a passively secure PKE, wihch we called the "encrypt-then-MAC" transformation. When applied to the CPA subroutines of ML-KEM, the resulting KEM (which we call one-time ML-KEM) reduced decapsulation CPU cycles by more than 70%, while incurring less than 10% penalty in encapsulation. Our KEM transformation (and other 1CCA secure KEMs) is suitable for ephemeral key exchange in TLS 1.3 and KEMTLS.

In Chapter 4 we discussed how we generated post-quantum certificate chains and implemented PQ-TLS/KEMTLS client/server, including a baremetal TCP client stack. Our implementation is the first to converge all dependencies onto a single open-source TLS

library (WolfSSL), so future works evaluating PQ-TLS/KEMTLS can have an easier time managing different external libraries.

Finally, Chapter 5 presents detailed latency metrics for PQ-TLS/KEMTLS handshake between an embedded client and a desktop server. The data not only proved feasibility of both protocols at all NIST security levels, we also drew some practical recommendations for real-world deloyments, such as using 1CCA KEM for ephemeral key exchange and prefer Falcon or ML-DSA for offline signatures.

6.1 Open problems and future works

6.1.1 Authenticating embedded device

We did not test mutual authentication on the embedded client, nor did we test the embedded device as a TLS server, because both scenarios require authenticating an embedded device. The main concern with authenticating an embedded device is the safety of the secret key. Embedded devices often operate in hostile environments where the risks of physical instrusion and sophisticated fault attacks are non-trival. While many platforms include some "secure boot" features that can increase the cost of dumping firmware and extracting secrets, they are not impossible to defeat.

On the other hand, if there is a way to safely store secrets (e.g. with a TPM), then the embedded device can use TLS in PSK mode, which is far more efficient because authentication is done with symmetric primitives instead of public-key cryptography. In summary, the scenario in which authenticating an embedded device with public-key primitives remains to be found.

6.1.2 Using Classic McEliece

HQC is the only code-based KEM we tested and shows abysmal performance. We suspect that Classic McEliece may be a better fit, such as in KEMTLS-PDK. This is because an embedded client will likely only talk to a few servers, so the long-term public keys can be pre-loaded into the firmware, thus mitigating the cost of transporting Classic McEliece's large public keys.

Another concern is the feasibility of loading the public key into memory: the smallest of McEliece public key uses more than 250KB, which is bigger than many microcontrollers

have RAM. However, because McEliece encryption is a single matrix-multiplication, the public key can be partially loaded into RAM. Streaming McEliece public key has been shown to be feasible Bernstein et al. [15].

References

[1] Martin R. Albrecht, Christian Hanser, Andrea Höller, Thomas Pöppelmann, Fernando Virdia, and Andreas Wallner. Implementing rlwe-based schemes using an RSA co-processor. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2019(1):169–208, 2019.

[2] Nouri Alnahawi, Johannes Müller, Jan Oupický, and Alexander Wiesmaier. Sok: Post-quantum TLS handshake. *IACR Cryptol. ePrint Arch.*, page 1873, 2023.

[3] Nouri Alnahawi, Johannes Müller, Jan Oupický, and Alexander Wiesmaier. A comprehensive survey on post-quantum TLS. *IACR Commun. Cryptol.*, 1(2):6, 2024.

[4] Dorian Amiet, Andreas Curiger, and Paul Zbinden. Fpga-based accelerator for post-quantum signature scheme SPHINCS-256. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(1):18–39, 2018.

[5] Nimrod Aviram, Benjamin Dowling, Ilan Komargodski, Kenneth G. Paterson, Eyal Ronen, and Eylon Yogev. Practical (post-quantum) key combiners from one-wayness and applications to TLS. *IACR Cryptol. ePrint Arch.*, page 65, 2022.

[6] Utsav Banerjee and Anantha P. Chandrakasan. Efficient post-quantum TLS handshakes using identity-based key exchange from lattices. In *2020 IEEE International Conference on Communications, ICC 2020, Dublin, Ireland, June 7-11, 2020*, pages 1–6. IEEE, 2020.

[7] Utsav Banerjee, Siddharth Das, and Anantha P. Chandrakasan. Accelerating post-quantum cryptography using an energy-efficient TLS crypto-processor. In *IEEE International Symposium on Circuits and Systems, ISCAS 2020, Sevilla, Spain, October 10-21, 2020*, pages 1–5. IEEE, 2020.

[8] Jon Barton, William J. Buchanan, Nikolaos Pitropakis, Sarwar Sayeed, and Will Abramson. Post quantum cryptography analysis of TLS tunneling on a constrained device. In Paolo Mori, Gabriele Lenzini, and Steven Furnell, editors, *Proceedings of the 8th International Conference on Information Systems Security and Privacy, ICISSP 2022, Online Streaming, February 9-11, 2022*, pages 551–561. SCITEPRESS, 2022.

[9] Mihir Bellare and Phillip Rogaway. Entity authentication and key distribution. In Douglas R. Stinson, editor, *Advances in Cryptology - CRYPTO '93, 13th Annual International Cryptology Conference, Santa Barbara, California, USA, August 22-26, 1993, Proceedings*, volume 773 of *Lecture Notes in Computer Science*, pages 232–249. Springer, 1993.

[10] Mihir Bellare and Phillip Rogaway. Optimal asymmetric encryption. In Alfredo De Santis, editor, *Advances in Cryptology - EUROCRYPT '94, Workshop on the Theory and Application of Cryptographic Techniques, Perugia, Italy, May 9-12, 1994, Proceedings*, volume 950 of *Lecture Notes in Computer Science*, pages 92–111. Springer, 1994.

[11] Daniel J. Bernstein. The poly1305-aes message-authentication code. In Henri Gilbert and Helena Handschuh, editors, *Fast Software Encryption: 12th International Workshop, FSE 2005, Paris, France, February 21-23, 2005, Revised Selected Papers*, volume 3557 of *Lecture Notes in Computer Science*, pages 32–49. Springer, 2005.

[12] Daniel J. Bernstein. FO derandomization sometimes damages security. Cryptology ePrint Archive, Paper 2021/912, 2021.

[13] Daniel J. Bernstein, Billy Bob Brumley, Ming-Shing Chen, and Nicola Tuveri. OpenSSLNTRU: Faster post-quantum TLS key exchange. *CoRR*, abs/2106.08759, 2021.

[14] Daniel J. Bernstein, Billy Bob Brumley, Ming-Shing Chen, and Nicola Tuveri. OpenSSLNTRU: Faster post-quantum TLS key exchange. In Kevin R. B. Butler and Kurt Thomas, editors, *31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10-12, 2022*, pages 845–862. USENIX Association, 2022.

[15] Daniel J. Bernstein and Tanja Lange. Mctiny: Fast high-confidence post-quantum key erasure for tiny network servers. In Srdjan Capkun and Franziska Roesner,

editors, *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*, pages 1731–1748. USENIX Association, 2020.

[16] Daniel J. Bernstein and Tanja Lange. eBACS: ECRYPT Benchmarking of Cryptographic Systems. https://bench.cr.yp.to, 2025. Accessed: 26 June 2025.

[17] John Black and Phillip Rogaway. CBC macs for arbitrary-length messages: The three-key constructions. *J. Cryptol.*, 18(2):111–131, 2005.

[18] Bruno Blanchet and Charlie Jacomme. Post-quantum sound cryptoverif and verification of hybrid TLS and SSH key-exchanges. In *37th IEEE Computer Security Foundations Symposium, CSF 2024, Enschede, Netherlands, July 8-12, 2024*, pages 543–556. IEEE, 2024.

[19] Joppe W. Bos, Craig Costello, Michael Naehrig, and Douglas Stebila. Post-quantum key exchange for the TLS protocol from the ring learning with errors problem. *IACR Cryptol. ePrint Arch.*, page 599, 2014.

[20] Joppe W. Bos, Craig Costello, Michael Naehrig, and Douglas Stebila. Post-quantum key exchange for the TLS protocol from the ring learning with errors problem. In *2015 IEEE Symposium on Security and Privacy, SP 2015, San Jose, CA, USA, May 17-21, 2015*, pages 553–570. IEEE Computer Society, 2015.

[21] Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS - kyber: A cca-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy, EuroS&P 2018, London, United Kingdom, April 24-26, 2018*, pages 353–367. IEEE, 2018.

[22] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. *SIGACT News*, 28(2):14–19, 1997.

[23] Kevin Bürstinghaus-Steinbach, Christoph Krauß, Ruben Niederhagen, and Michael Schneider. Post-quantum TLS on embedded systems. *IACR Cryptol. ePrint Arch.*, page 308, 2020.

[24] Kevin Bürstinghaus-Steinbach, Christoph Krauß, Ruben Niederhagen, and Michael Schneider. Post-quantum TLS on embedded systems: Integrating and evaluating kyber and SPHINCS+ with mbed TLS. In Hung-Min Sun, Shiuh-Pyng Shieh, Guofei Gu, and Giuseppe Ateniese, editors, *ASIA CCS '20: The 15th ACM Asia Conference*

on Computer and Communications Security, Taipei, Taiwan, October 5-9, 2020, pages 841–852. ACM, 2020.

[25] Fabio Campos, Jorge Chávez-Saab, Jesús-Javier Chi-Domínguez, Michael Meyer, Krijn Reijnders, Francisco Rodríguez-Henríquez, Peter Schwabe, and Thom Wiggers. On the practicality of post-quantum TLS using large-parameter CSIDH. *IACR Cryptol. ePrint Arch.*, page 793, 2023.

[26] Sofía Celi, Armando Faz-Hernández, Nick Sullivan, Goutam Tamvada, Luke Valenta, Thom Wiggers, Bas Westerbaan, and Christopher A. Wood. Implementing and measuring KEMTLS. In Patrick Longa and Carla Ràfols, editors, *Progress in Cryptology - LATINCRYPT 2021 - 7th International Conference on Cryptology and Information Security in Latin America, Bogotá, Colombia, October 6-8, 2021, Proceedings*, volume 12912 of *Lecture Notes in Computer Science*, pages 88–107. Springer, 2021.

[27] Sofía Celi, Jonathan Hoyland, Douglas Stebila, and Thom Wiggers. A tale of two models: Formal verification of KEMTLS via tamarin. In Vijayalakshmi Atluri, Roberto Di Pietro, Christian Damsgaard Jensen, and Weizhi Meng, editors, *Computer Security - ESORICS 2022 - 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26-30, 2022, Proceedings, Part III*, volume 13556 of *Lecture Notes in Computer Science*, pages 63–83. Springer, 2022.

[28] Ronald Cramer, Goichiro Hanaoka, Dennis Hofheinz, Hideki Imai, Eike Kiltz, Rafael Pass, Abhi Shelat, and Vinod Vaikuntanathan. Bounded CCA2-secure encryption. In Kaoru Kurosawa, editor, *Advances in Cryptology - ASIACRYPT 2007, 13th International Conference on the Theory and Application of Cryptology and Information Security, Kuching, Malaysia, December 2-6, 2007, Proceedings*, volume 4833 of *Lecture Notes in Computer Science*, pages 502–518. Springer, 2007.

[29] Eric Crockett, Christian Paquin, and Douglas Stebila. Prototyping post-quantum and hybrid key exchange and authentication in TLS and SSH. *IACR Cryptol. ePrint Arch.*, page 858, 2019.

[30] Alexander W. Dent. A designer's guide to kems. In Kenneth G. Paterson, editor, *Cryptography and Coding, 9th IMA International Conference, Cirencester, UK, December 16-18, 2003, Proceedings*, volume 2898 of *Lecture Notes in Computer Science*, pages 133–151. Springer, 2003.

[31] Tim Dierks and Eric Rescorla. The transport layer security (TLS) protocol version 1.2. *RFC*, 5246:1–104, 2008.

[32] Ronny Döring and Marc Geitz. Post-quantum cryptography in use: Empirical analysis of the TLS handshake performance. In *2022 IEEE/IFIP Network Operations and Management Symposium, NOMS 2022, Budapest, Hungary, April 25-29, 2022*, pages 1–5. IEEE, 2022.

[33] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol candidates. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015*, pages 1197–1210. ACM, 2015.

[34] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 draft-10 full and pre-shared key handshake protocol. *IACR Cryptol. ePrint Arch.*, page 81, 2016.

[35] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol. *J. Cryptol.*, 34(4):37, 2021.

[36] Adam Dunkels. Design and implementation of the lwip tcp/ip stack. *Swedish Institute of Computer Science*, 2, 03 2001.

[37] Cynthia Dwork, Moni Naor, and Omer Reingold. Immunizing encryption schemes from decryption errors. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology - EUROCRYPT 2004, International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004, Proceedings*, volume 3027 of *Lecture Notes in Computer Science*, pages 342–360. Springer, 2004.

[38] Marc Fischlin and Felix Günther. Multi-stage key exchange and the case of google's QUIC protocol. In Gail-Joon Ahn, Moti Yung, and Ninghui Li, editors, *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, Scottsdale, AZ, USA, November 3-7, 2014*, pages 1193–1204. ACM, 2014.

[39] Marc Fischlin and Felix Günther. Replay attacks on zero round-trip time: The case of the TLS 1.3 handshake candidates. In *2017 IEEE European Symposium on Security and Privacy, EuroS&P 2017, Paris, France, April 26-28, 2017*, pages 60–75. IEEE, 2017.

[40] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *Advances in Cryptology*

- *CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 537–554. Springer, 1999.

[41] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *J. Cryptol.*, 26(1):80–101, 2013.

[42] Eiichiro Fujisaki, Tatsuaki Okamoto, David Pointcheval, and Jacques Stern. RSA-OAEP is secure under the RSA assumption. In Joe Kilian, editor, *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings*, volume 2139 of *Lecture Notes in Computer Science*, pages 260–274. Springer, 2001.

[43] Xinwei Gao, Jintai Ding, Lin Li, Saraswathy RV, and Jiqiang Liu. Efficient implementation of password-based authenticated key exchange from RLWE and post-quantum TLS. *IACR Cryptol. ePrint Arch.*, page 1192, 2017.

[44] Xinwei Gao, Jintai Ding, Lin Li, Saraswathy RV, and Jiqiang Liu. Efficient implementation of password-based authenticated key exchange from RLWE and post-quantum TLS. *Int. J. Netw. Secur.*, 20(5):923–930, 2018.

[45] Xinwei Gao, Lin Li, Jintai Ding, Jiqiang Liu, R. V. Saraswathy, and Zhe Liu. Fast discretized gaussian sampling and post-quantum TLS ciphersuite. In Joseph K. Liu and Pierangela Samarati, editors, *Information Security Practice and Experience - 13th International Conference, ISPEC 2017, Melbourne, VIC, Australia, December 13-15, 2017, Proceedings*, volume 10701 of *Lecture Notes in Computer Science*, pages 551–565. Springer, 2017.

[46] Carlos Rubio Garcia, Abraham Cano Aguilera, Juan Jose Vegas Olmos, Idelfonso Tafur Monroy, and Simon Rommel. Quantum-resistant TLS 1.3: A hybrid solution combining classical, quantum and post-quantum cryptography. In *28th IEEE International Workshop on Computer Aided Modeling and Design of Communication Links and Networks , CAMAD 2023, Edinburgh, United Kingdom, November 6-8, 2023*, pages 246–251. IEEE, 2023.

[47] Alexandre Augusto Giron, João Pedro Adami do Nascimento, Ricardo Custódio, Lucas Pandolfo Perin, and Víctor Mateu. Post-quantum hybrid KEMTLS performance in simulated and real network environments. In Abdelrahaman Aly and Mehdi Tibouchi, editors, *Progress in Cryptology - LATINCRYPT 2023 - 8th International Conference on Cryptology and Information Security in Latin America,*

LATINCRYPT 2023, Quito, Ecuador, October 3-6, 2023, Proceedings, volume 14168 of *Lecture Notes in Computer Science*, pages 293–312. Springer, 2023.

[48] Shafi Goldwasser and Silvio Micali. Probabilistic encryption and how to play mental poker keeping secret all partial information. In Harry R. Lewis, Barbara B. Simons, Walter A. Burkhard, and Lawrence H. Landweber, editors, *Proceedings of the 14th Annual ACM Symposium on Theory of Computing, May 5-7, 1982, San Francisco, California, USA*, pages 365–377. ACM, 1982.

[49] Ruben Gonzalez and Thom Wiggers. KEMTLS vs. post-quantum TLS: performance on embedded systems. In Lejla Batina, Stjepan Picek, and Mainack Mondal, editors, *Security, Privacy, and Applied Cryptography Engineering - 12th International Conference, SPACE 2022, Jaipur, India, December 9-12, 2022, Proceedings*, volume 13783 of *Lecture Notes in Computer Science*, pages 99–117. Springer, 2022.

[50] Joint Task Force Transformation Initiative Interagency Working Group. Security and privacy controls for federal information systems and organizations. Technical Report NIST Special Publication (SP) 800-53, Rev. 4, Includes updates as of January 22, 2015, National Institute of Standards and Technology, Gaithersburg, MD, 2013.

[51] Lov K. Grover. A fast quantum mechanical algorithm for database search. In Gary L. Miller, editor, *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*, pages 212–219. ACM, 1996.

[52] Felix Günther, Simon Rastikian, Patrick Towa, and Thom Wiggers. KEMTLS with delayed forward identity protection in (almost) a single round trip. In Giuseppe Ateniese and Daniele Venturi, editors, *Applied Cryptography and Network Security - 20th International Conference, ACNS 2022, Rome, Italy, June 20-23, 2022, Proceedings*, volume 13269 of *Lecture Notes in Computer Science*, pages 253–272. Springer, 2022.

[53] Yacoub Hanna, Diana Pineda, Maryna Veksler, Manish Paudel, Kemal Akkaya, Mila Anastasova, and Reza Azarderakhsh. Integrating post-quantum TLS into the control plane of 5g networks. In *IEEE International Performance, Computing, and Communications Conference, IPCCC 2024, Orlando, FL, USA, November 22-24, 2024*, pages 1–8. IEEE, 2024.

[54] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD,*

USA, November 12-15, 2017, Proceedings, Part I, volume 10677 of *Lecture Notes in Computer Science*, pages 341–371. Springer, 2017.

[55] Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Failing gracefully: Decryption failures and the fujisaki-okamoto transform. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part IV*, volume 13794 of *Lecture Notes in Computer Science*, pages 414–443. Springer, 2022.

[56] James Howe, Tobias Oder, Markus Krausz, and Tim Güneysu. Standard lattice-based key encapsulation on embedded devices. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(3):372–393, 2018.

[57] Loïs Huguenin-Dumittan and Serge Vaudenay. On ind-qcca security in the ROM and its applications - CPA security is sufficient for TLS 1.3. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 613–642. Springer, 2022.

[58] Tetsu Iwata and Kaoru Kurosawa. OMAC: one-key CBC MAC. In Thomas Johansson, editor, *Fast Software Encryption, 10th International Workshop, FSE 2003, Lund, Sweden, February 24-26, 2003, Revised Papers*, volume 2887 of *Lecture Notes in Computer Science*, pages 129–153. Springer, 2003.

[59] Panos Kampanakis and Will Childs-Klein. The impact of data-heavy, post-quantum TLS 1.3 on the time-to-last-byte of real-world connections. *IACR Cryptol. ePrint Arch.*, page 176, 2024.

[60] Panos Kampanakis and Michael G. Kallitsis. Faster post-quantum TLS handshakes without intermediate CA certificates. In Shlomi Dolev, Jonathan Katz, and Amnon Meisels, editors, *Cyber Security, Cryptology, and Machine Learning - 6th International Symposium, CSCML 2022, Be'er Sheva, Israel, June 30 - July 1, 2022, Proceedings*, volume 13301 of *Lecture Notes in Computer Science*, pages 337–355. Springer, 2022.

[61] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking NIST PQC on ARM cortex-m4. *IACR Cryptol. ePrint Arch.*, page 844, 2019.

[62] Matthias J. Kannwischer, Peter Schwabe, Douglas Stebila, and Thom Wiggers. Improving software quality in cryptography standardization projects. In *IEEE European Symposium on Security and Privacy, EuroS&P 2022 - Workshops, Genoa, Italy, June 6-10, 2022*, pages 19–30. IEEE, 2022.

[63] Brian Koziel, Reza Azarderakhsh, and Mehran Mozaffari Kermani. A high-performance and scalable hardware architecture for isogeny-based cryptography. *IEEE Trans. Computers*, 67(11):1594–1609, 2018.

[64] Hugo Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. In Tal Rabin, editor, *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings*, volume 6223 of *Lecture Notes in Computer Science*, pages 631–648. Springer, 2010.

[65] Hugo Krawczyk and Pasi Eronen. Hmac-based extract-and-expand key derivation function (HKDF). *RFC*, 5869:1–14, 2010.

[66] Hugo Krawczyk and Hoeteck Wee. The OPTLS protocol and TLS 1.3. In *IEEE European Symposium on Security and Privacy, EuroS&P 2016, Saarbrücken, Germany, March 21-24, 2016*, pages 81–96. IEEE, 2016.

[67] L LAMPORT. Constructing digital signatures from a one-way function. *Report SRI Intl. CSL 98*, 1979.

[68] Vadim Lyubashevsky. Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*, volume 5912 of *Lecture Notes in Computer Science*, pages 598–616. Springer, 2009.

[69] Dimitri Mankowski, Thom Wiggers, and Veelasha Moonsamy. TLS → post-quantum TLS: inspecting the TLS landscape for PQC adoption on android. In *IEEE European Symposium on Security and Privacy, EuroS&P 2023 - Workshops, Delft, Netherlands, July 3-7, 2023*, pages 526–538. IEEE, 2023.

[70] Dimitri Mankowski, Thom Wiggers, and Veelasha Moonsamy. TLS â†' post-quantum TLS: inspecting the TLS landscape for PQC adoption on android. *IACR Cryptol. ePrint Arch.*, page 734, 2023.

[71] Dominik Marchsreiter and Johanna Sepúlveda. Hybrid post-quantum enhanced TLS 1.3 on embedded devices. In *25th Euromicro Conference on Digital System Design,*

DSD 2022, Maspalomas, Spain, August 31 - Sept. 2, 2022, pages 905–912. IEEE, 2022.

[72] David A. McGrew and John Viega. The security and performance of the galois/-counter mode (GCM) of operation. In Anne Canteaut and Kapalee Viswanathan, editors, *Progress in Cryptology - INDOCRYPT 2004, 5th International Conference on Cryptology in India, Chennai, India, December 20-22, 2004, Proceedings*, volume 3348 of *Lecture Notes in Computer Science*, pages 343–355. Springer, 2004.

[73] Callum McLoughlin, Clémentine Gritti, and Juliet Samandari. Full post-quantum datagram TLS handshake in the internet of things. In Said El Hajji, Sihem Mesnager, and El Mamoun Souidi, editors, *Codes, Cryptology and Information Security - 4th International Conference, C2SI 2023, Rabat, Morocco, May 29-31, 2023, Proceedings*, volume 13874 of *Lecture Notes in Computer Science*, pages 57–76. Springer, 2023.

[74] Simon Meier, Benedikt Schmidt, Cas Cremers, and David A. Basin. The TAMARIN prover for the symbolic analysis of security protocols. In Natasha Sharygina and Helmut Veith, editors, *Computer Aided Verification - 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13-19, 2013. Proceedings*, volume 8044 of *Lecture Notes in Computer Science*, pages 696–701. Springer, 2013.

[75] National Institute of Standards and Technology. Security (evaluation criteria) — post-quantum cryptography standardization, 2017. Accessed: 2025-06-24.

[76] National Institute of Standards and Technology. Sha-3 standard: Permutation-based hash and extendable-output functions. Technical Report Federal Information Processing Standards Publication (FIPS) NIST FIPS 202, U.S. Department of Commerce, Washington, D.C., 2015.

[77] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Technical Report Federal Information Processing Standards Publication (FIPS) NIST FIPS 203, U.S. Department of Commerce, Washington, D.C., 2024.

[78] Christian Paquin, Douglas Stebila, and Goutam Tamvada. Benchmarking post-quantum cryptography in TLS. *IACR Cryptol. ePrint Arch.*, page 1447, 2019.

[79] Christian Paquin, Douglas Stebila, and Goutam Tamvada. Benchmarking post-quantum cryptography in TLS. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020, Paris,*

France, April 15-17, 2020, Proceedings, volume 12100 of *Lecture Notes in Computer Science*, pages 72–91. Springer, 2020.

[80] Sebastian Paul, Yulia Kuzovkova, Norman Lahr, and Ruben Niederhagen. Mixed certificate chains for the transition to post-quantum authentication in TLS 1.3. *IACR Cryptol. ePrint Arch.*, page 1447, 2021.

[81] Sebastian Paul, Yulia Kuzovkova, Norman Lahr, and Ruben Niederhagen. Mixed certificate chains for the transition to post-quantum authentication in TLS 1.3. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIA CCS '22: ACM Asia Conference on Computer and Communications Security, Nagasaki, Japan, 30 May 2022 - 3 June 2022*, pages 727–740. ACM, 2022.

[82] Sebastian Paul, Felix Schick, and Jan Seedorf. Tpm-based post-quantum cryptography: A case study on quantum-resistant and mutually authenticated TLS for iot environments. In Delphine Reinhardt and Tilo Müller, editors, *ARES 2021: The 16th International Conference on Availability, Reliability and Security, Vienna, Austria, August 17-20, 2021*, pages 3:1–3:10. ACM, 2021.

[83] Prasanna Ravi, Sujoy Sinha Roy, Anupam Chattopadhyay, and Shivam Bhasin. Generic side-channel attacks on cca-secure lattice-based PKE and KEM schemes. *IACR Cryptol. ePrint Arch.*, page 948, 2019.

[84] Eric Rescorla. The transport layer security (TLS) protocol version 1.3. *RFC*, 8446:1–160, 2018.

[85] Kadir Sabanci and Mumin Cebe. Exploring post-quantum cryptographic schemes for TLS in 5g nb-iot: Feasibility and recommendations. In Paul A. Pavlou, Vishal Midha, Animesh Animesh, Traci A. Carte, Alexandre R. Graeml, and Alanah Mitchell, editors, *29th Americas Conference on Information Systems, AMCIS 2023, Panama City, Panama, August 10-12, 2023*. Association for Information Systems, 2023.

[86] Kadir Sabanci and Mumin Cebe. Exploring post-quantum cryptographic schemes for TLS in 5g nb-iot: Feasibility and recommendations. *CoRR*, abs/2309.03338, 2023.

[87] Maximilian Schöffel, Frederik Lauer, Carl Christian Rheinländer, and Norbert Wehn. On the energy costs of post-quantum kems in tls-based low-power secure iot. In *IoTDI '21: International Conference on Internet-of-Things Design and Implementation, Virtual Event / Charlottesville, VA, USA, May 18-21, 2021*, pages 158–168. ACM, 2021.

[88] Peter Schwabe, Douglas Stebila, and Thom Wiggers. Post-quantum TLS without handshake signatures. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 1461–1480. ACM, 2020.

[89] Peter Schwabe, Douglas Stebila, and Thom Wiggers. More efficient post-quantum KEMTLS with pre-distributed public keys. In Elisa Bertino, Haya Schulmann, and Michael Waidner, editors, *Computer Security - ESORICS 2021 - 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4-8, 2021, Proceedings, Part I*, volume 12972 of *Lecture Notes in Computer Science*, pages 3–22. Springer, 2021.

[90] Michael Scott. On TLS for the internet of things, in a post quantum world. *IACR Cryptol. ePrint Arch.*, page 95, 2023.

[91] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, 20-22 November 1994*, pages 124–134. IEEE Computer Society, 1994.

[92] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM J. Comput.*, 26(5):1484–1509, 1997.

[93] Victor Shoup. OAEP reconsidered. In Joe Kilian, editor, *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings*, volume 2139 of *Lecture Notes in Computer Science*, pages 239–259. Springer, 2001.

[94] Dimitrios Sikeridis, Sean Huntley, David Ott, and Michael Devetsikiotis. Intermediate certificate suppression in post-quantum TLS: an approximate membership querying approach. In Giuseppe Bianchi and Alessandro Mei, editors, *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies, CoNEXT 2022, Roma, Italy, December 6-9, 2022*, pages 35–42. ACM, 2022.

[95] Dimitrios Sikeridis, Sean Huntley, David Ott, and Michael Devetsikiotis. Intermediate certificate suppression in post-quantum TLS: an approximate membership querying approach. *IACR Cryptol. ePrint Arch.*, page 1556, 2022.

[96] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH. In Dongsu Han and

Anja Feldmann, editors, *CoNEXT '20: The 16th International Conference on emerging Networking EXperiments and Technologies, Barcelona, Spain, December, 2020*, pages 149–156. ACM, 2020.

[97] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Post-quantum authentication in TLS 1.3: A performance study. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society, 2020.

[98] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Post-quantum authentication in TLS 1.3: A performance study. *IACR Cryptol. ePrint Arch.*, page 71, 2020.

[99] Markus Sosnowski, Florian Wiedner, Eric Hauser, Lion Steger, Dimitrios Schoinianakis, Sebastian Gallenmüller, and Georg Carle. The performance of post-quantum TLS 1.3. In Dario Rossi, Stefano Secci, Olivier Bonaventure, and Lili Qiu, editors, *Companion of the 19th International Conference on emerging Networking EXperiments and Technologies, CoNEXT 2023 (Short Papers), Paris, France, December 5-8, 2023*, pages 19–27. ACM, 2023.

[100] Douglas Stebila and Michele Mosca. Post-quantum key exchange for the internet and the open quantum safe project. In Roberto Avanzi and Howard M. Heys, editors, *Selected Areas in Cryptography - SAC 2016 - 23rd International Conference, St. John's, NL, Canada, August 10-12, 2016, Revised Selected Papers*, volume 10532 of *Lecture Notes in Computer Science*, pages 14–37. Springer, 2016.

[101] Yutaro Tanaka, Rei Ueno, Keita Xagawa, Akira Ito, Junko Takahashi, and Naofumi Homma. Multiple-valued plaintext-checking side-channel attacks on post-quantum kems. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(3):473–503, 2023.

[102] George Tasopoulos, Charis Dimopoulos, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Energy consumption evaluation of post-quantum TLS 1.3 for resource-constrained embedded devices. *IACR Cryptol. ePrint Arch.*, page 506, 2023.

[103] George Tasopoulos, Charis Dimopoulos, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Energy consumption evaluation of post-quantum TLS 1.3 for resource-constrained embedded devices. In Andrea Bartolini, Kristian F. D. Rietveld, Catherine D. Schuman, and Jose Moreira, editors, *Proceedings of*

the 20th ACM International Conference on Computing Frontiers, CF 2023, Bologna, Italy, May 9-11, 2023, pages 366–374. ACM, 2023.

[104] George Tasopoulos, Jinhui Li, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum TLS 1.3 on embedded systems. *IACR Cryptol. ePrint Arch.*, page 1553, 2021.

[105] George Tasopoulos, Jinhui Li, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum TLS 1.3 on resource-constrained embedded systems. Cryptology ePrint Archive, Paper 2021/1553, 2021.

[106] George Tasopoulos, Jinhui Li, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum TLS 1.3 on resource-constrained embedded systems. In Chunhua Su, Dimitris Gritzalis, and Vincenzo Piuri, editors, *Information Security Practice and Experience - 17th International Conference, ISPEC 2022, Taipei, Taiwan, November 23-25, 2022, Proceedings*, volume 13620 of *Lecture Notes in Computer Science*, pages 432–451. Springer, 2022.

[107] Duong Dinh Tran, Canh Minh Do, Santiago Escobar, and Kazuhiro Ogata. Hybrid post-quantum TLS formal specification in maude-npa - toward its security analysis. In Sedat Akleylek, Santiago Escobar, Kazuhiro Ogata, and Ayoub Otmani, editors, *Proceedings of the International Workshop on Formal Analysis and Verification of Post-Quantum Cryptographic Protocols co-located with the 23rd International Conference on Formal Engineering Methods (ICFEM 2022), Madrid, Spain, October 24, 2022*, volume 3280 of *CEUR Workshop Proceedings*, pages 50–64. CEUR-WS.org, 2022.

[108] Rei Ueno, Keita Xagawa, Yutaro Tanaka, Akira Ito, Junko Takahashi, and Naofumi Homma. Curse of re-encryption: A generic power/em analysis on post-quantum kems. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(1):296–322, 2022.

[109] Mark N. Wegman and Larry Carter. New hash functions and their use in authentication and set equality. *J. Comput. Syst. Sci.*, 22(3):265–279, 1981.

[110] Bas Westerbaan. The state of the post-quantum internet, Mar 2024.

[111] Jieyu Zheng, Haoliang Zhu, Yifan Dong, Zhenyu Song, Zhenhao Zhang, Yafang Yang, and Yunlei Zhao. Faster post-quantum TLS 1.3 based on ML-KEM: implementation and assessment. In Joaquín García-Alfaro, Rafal Kozik, Michal Choras, and

Sokratis K. Katsikas, editors, *Computer Security - ESORICS 2024 - 29th European Symposium on Research in Computer Security, Bydgoszcz, Poland, September 16-20, 2024, Proceedings, Part II*, volume 14983 of *Lecture Notes in Computer Science*, pages 123–143. Springer, 2024.

[112] Biming Zhou, Haodong Jiang, and Yunlei Zhao. CPA-secure KEMs are also sufficient for post-quantum TLS 1.3. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part III*, volume 15486 of *Lecture Notes in Computer Science*, pages 433–464. Springer, 2024.

APPENDICES

Appendix A

Cryptography definitions

A.1 Public-key encryption scheme

Syntax A public-key encryption scheme (PKE) is a collection of three routines $(\texttt{KeyGen}, \texttt{Enc}, \texttt{Dec})$ defined over some plaintext space $\mathcal{M}$ and some ciphertext space $\mathcal{C}$. Key generation $(\texttt{pk}, \texttt{sk}) \xleftarrow{\$} \texttt{KeyGen}(1^\lambda)$ is a randomized routine that returns a keypair consisting of a public encryption key and a secret decryption key. The encryption routine $\texttt{Enc} : (\texttt{pk}, m) \mapsto c$ encrypts the input plaintext m under the input public key $\texttt{pk}$ and produces a ciphertext c. The decryption routine $\texttt{Dec} : (\texttt{sk}, c) \mapsto m$ decrypts the input ciphertext c under the input secret key and produces the corresponding plaintext. Where the encryption routine is randomized, we denote the randomness by a coin $r \in \mathcal{R}$ where $\mathcal{R}$ is called the coin space. Decryption routines are assumed to always be deterministic.

Correctness A PKE is δ-correct if

$$E\left[\max_{m \in \mathcal{M}} P\left[\texttt{Dec}(\texttt{sk}, c) \neq m \mid c \xleftarrow{\$} \texttt{Enc}(\texttt{pk}, m)\right]\right] \leq \delta$$

Where the expectation is taken with respect to the probability distribution of all possible keypairs. For many lattice-based cryptosystems, decryption failures could leak information about the secret key, although the probability of a decryption failure is low enough that classical adversaries cannot exploit decryption failure more than they can defeat the underlying lattice problems.

Security The security of PKE's is conventionally discussed using adversarial games played between a challenger and an adversary [48]. In the OW-ATK game (Figure A.1), the challenger samples a random keypair and a random encryption. The adversary is given the public key, the random encryption (also called the challenge ciphertext), and access to ATK, then asked to decrypt the challenge ciphertext.

OW-ATK game

1: $(\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}(1^\lambda)$
2: $m^* \xleftarrow{\$} \mathcal{M}$
3: $c^* \xleftarrow{\$} \mathtt{Enc}(\mathtt{pk}, m)$
4: $\hat{m} \xleftarrow{\$} A^{\mathrm{ATK}}(1^\lambda, \mathtt{pk}, c^*)$
5: **return** $[\![\hat{m} = m^*]\!]$

Figure A.1: The one-wayness game: challenger samples a random keypair and a random encryption, and the adversary wins if it correctly produces the decryption

The advantage of an adversary is its probability of producing the correct decryption: $\mathrm{Adv}^{\text{OW-ATK}}_{\mathtt{PKE}}(A) = P[\hat{m} = m^*]$. A PKE is said to be OW-ATK secure if no efficient adversary can win the OW-ATK game with non-negligible probability.

IND-ATK game

1: $(\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}(1^\lambda)$
2: $(m_0, m_1) \xleftarrow{\$} A^{\mathrm{ATK}}(1^\lambda, \mathtt{pk})$
3: $b \xleftarrow{\$} \{0, 1\}$
4: $c^* \xleftarrow{\$} \mathtt{Enc}(\mathtt{pk}, m_b)$
5: $\hat{b} \xleftarrow{\$} A^{\mathrm{ATK}}(1^\lambda, \mathtt{pk}, c^*)$
6: **return** $[\![\hat{b} = b]\!]$

Figure A.2: IND-ATK game: adversary is asked to distinguish the encryption of one message from another

In the IND-ATK game (Figure A.2), the adversary chooses two distinct messages and receives the encryption of one of them, randomly selected by the challenger. The advantage of an adversary is its probability of correctly distinguishing the ciphertext of one message

from the other beyond blind guess: $\mathrm{Adv}^{\text{IND-ATK}}_{\mathtt{PKE}}(A) = |P[\hat{b} = b] - \frac{1}{2}|$. A PKE is said to be IND-ATK secure if no efficient adversary can win the IND-ATK game with non-negligible advantage.

$\mathcal{O}^{\mathtt{Dec}}(c)$

1: **return** $\mathtt{Dec}(\mathtt{sk}, c)$

$\mathcal{O}^{\mathtt{PCO}}(m, c)$

1: **return** $[\![\mathtt{Dec}(\mathtt{sk}, c) = m]\!]$

Figure A.3: Decryption oracle $\mathcal{O}^{\mathtt{Dec}}$ (left) answers decryption queries by returning the decryption of the queried ciphertext. Plaintext-checking oracle $\mathcal{O}^{\mathtt{PCO}}$ (right) answers whether the queried plaintext is the decryption of the queried ciphertext.

In public-key cryptography, all adversaries are assumed to have access to the public key (ATK = CPA). If the adversary has access to a decryption oracle, it is said to mount chosen-ciphertext attack (ATK = CCA). If the adversary has access to a plaintext-checking oracle (PCO) , then it is said to mount plaintext-checking attack (ATK = PCA). See Figure A.3 for algorithmic description of the oracles.

$$\text{ATK} = \begin{cases} \text{CPA} & \mathcal{O}^{\text{ATK}} = \cdot \\ \text{PCA} & \mathcal{O}^{\text{ATK}} = \mathcal{O}^{\mathtt{PCO}} \\ \text{CCA} & \mathcal{O}^{\text{ATK}} = \mathcal{O}^{\mathtt{Dec}} \end{cases}$$

A.2 Key encapsulation mechanism (KEM)

Syntax A key encapsulation mechanism (KEM) is a collection of three routines $(\mathtt{KeyGen}, \mathtt{Encap}, \mathtt{Decap})$ defined over some ciphertext space $\mathcal{C}$ and some key space $\mathcal{K}$. Key generation $\mathtt{KeyGen} : 1^\lambda \mapsto (\mathtt{pk}, \mathtt{sk})$ is a randomized routine that returns a keypair. Encapsulation $\mathtt{Encap} : \mathtt{pk} \mapsto (c, K)$ is a randomized routine that takes a public encapsulation key and returns a pair of ciphertext c and shared secret K (also commonly referred to as session key). Decapsulation $\mathtt{Decap} : (\mathtt{sk}, c) \mapsto K$ is a deterministic routine that uses the secret key $\mathtt{sk}$ to recover the shared secret K from the input ciphertext c. Where the KEM chooses to reject invalid ciphertext explicitly, the decapsulation routine can also output the rejection symbol $\perp$. We assume a KEM to be perfectly correct:

$$P\left[\mathtt{Decap}(\mathtt{sk}, c) = K \mid (\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}(1^\lambda); (c, K) \xleftarrow{\$} \mathtt{Encap}(\mathtt{pk})\right] = 1$$

Security Similar to PKE security, the security of KEM is discussed using adversarial games. In the IND-ATK game (Figure A.4), the challenger generates a random keypair and encapsulates a random secret; the adversary is given the public key and the ciphertext, then asked to distinguish the shared secret from a random bit string.

KEM IND-ATK Game

1: $(\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}((1^\lambda))$
2: $(c^*, K_0) \xleftarrow{\$} \mathtt{Encap}(\mathtt{pk})$
3: $K_1 \xleftarrow{\$} \mathcal{K}$
4: $b \xleftarrow{\$} \{0,1\}$
5: $\hat{b} \xleftarrow{\$} A^{\mathtt{ATK}}(1^\lambda, \mathtt{pk}, c^*, K_b)$
6: **return** $[\![\hat{b} = b]\!]$

Figure A.4: The IND-ATK game for KEM

The advantage of an adversary is its probability of winning beyong blind guess. A KEM is said to be IND-ATK secure if no efficient adversary can win the IND-ATK game with non-negligible advantage.

$$\mathtt{Adv}^{\mathtt{IND\text{-}ATK}}(A) = \left| P\left[A^{\mathtt{ATK}}(1^\lambda, c^*, K_b) = b \mid \begin{array}{r} (\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}(1^\lambda); \\ (c^*, K_0) \xleftarrow{\$} \mathtt{Encap}(\mathtt{pk}); \\ K_1 \xleftarrow{\$} \mathcal{K}; b \xleftarrow{\$} \{0,1\} \end{array} \right] - \frac{1}{2} \right|$$

By default, all adversaries are assumed to have the public key, with which they can mount chosen plaintext attacks (ATK = CPA). If the adversary as access to a decapsulation oracle $\mathcal{O}^{\mathtt{Decap}} : c \mapsto \mathtt{Decap}(\mathtt{sk}, c)$, it is said to mount a chosen-ciphertext attack (ATK = CCA).

A.3 Message authentication code (MAC)

Syntax A message authentication code (MAC) is a collection of two routines `Sign` and `Verify` defined over some key space $\mathcal{K}$, some message space $\mathcal{M}$, and some tag space $\mathcal{T}$. The signing routine $\mathtt{Sign} : (k, m) \mapsto t$ authenticates the message m under the symmetric

key k by producing a tag t. The verification routine $\texttt{Verify}(k, m, t)$ outputs 1 if the message-tag pair (m, t) is authentic under the symmetric key k and 0 otherwise. Many MAC constructions are deterministic: for these constructions it is simpler to denote the signing routine by $t \leftarrow \texttt{MAC}(k, m)$, and verification done using a simple comparison. Some MAC constructions require a distinct or randomized nonce $r \xleftarrow{\$} \mathcal{R}$, and the signing routine will take this additional argument $t \leftarrow \texttt{MAC}(k, m; r)$.

Security The standard security notion for a MAC is *existential unforgeability under chosen message attack (EUF-CMA)*. We define it using an adversarial game in which an adversary has access to a signing oracle $\mathcal{O}^{\texttt{Sign}} : m \mapsto \texttt{Sign}(k, m)$ and tries to produce a valid message-tag pair that has not been queried from the signing oracle (Figure A.5).

MAC EUF-CMA game

1: $k^* \xleftarrow{\$} \mathcal{K}$
2: $(\hat{m}, \hat{t}) \xleftarrow{\$} A^{\texttt{CMA}}()$
3: **return** $[\![\texttt{Verify}(k^*, \hat{m}, \hat{t}) \wedge (\hat{m}, \hat{t}) \notin \mathcal{O}^{\texttt{Sign}}]\!]$

Figure A.5: The signing oracle signs the queried message with the secret key. The adversary must produce a message-tag pair that has never been queried before

The advantage of the adversary is the probability that it successfully produces a valid message-tag pair. A MAC is said to be EUF-CMA secure if no efficient adversary has non-negligible advantage. Some MACs are *one-time existentially unforgeable* (we call them one-time MAC), meaning that each secret key can be used to authenticate exactly one message. The corresponding security game is identical to the EUF-CMA game except for that the signing oracle will only answer up to one query.

Appendix B

ML-KEM

Below are high-level summaries of the PKE sub-routines and the KEM routines of ML-KEM:

K-PKE.KeyGen()

1: $A \xleftarrow{\$} R_q^{k \times k}$
2: $\mathbf{s} \xleftarrow{\$} \mathcal{X}_{\eta_1}^k$
3: $\mathbf{e} \xleftarrow{\$} \mathcal{X}_{\eta_1}^k$
4: $\mathbf{t} \leftarrow A\mathbf{s} + \mathbf{e}$
5: $\mathtt{pk} \leftarrow (A, \mathbf{t})$
6: $\mathtt{sk} \leftarrow \mathbf{s}$
7: **return** $(\mathtt{pk}, \mathtt{sk})$

K-PKE.Enc(pk, m)

Ensure: $\mathtt{pk} = (A, \mathbf{t})$
Ensure: $m \in R_2$
1: $\mathbf{r} \xleftarrow{\$} \mathcal{X}_{\eta_1}^k$
2: $\mathbf{e}_1 \xleftarrow{\$} \mathcal{X}_{\eta_2}^k$
3: $e_2 \xleftarrow{\$} \mathcal{X}_{\eta_2}$
4: $\mathbf{c}_1 \leftarrow A\mathbf{r} + \mathbf{e}_1$
5: $c_2 \leftarrow \mathbf{t}^\intercal \mathbf{r} + e_2 + m \cdot \lfloor \frac{q}{2} \rceil$
6: **return** $(\mathbf{c}_1, c_2)$

K-PKE.Dec(sk, c)

Ensure: $c = (\mathbf{c}_1, c_2)$
Ensure: $\mathtt{sk} = \mathbf{s}$
1: $\hat{m} \leftarrow c_2 - \mathbf{c}_1^\intercal \cdot \mathbf{s}$
2: $\hat{m} \leftarrow \mathtt{Round}(\hat{m})$
3: **return** $\hat{m}$

Figure B.1: K-PKE is an IND-CPA secure PKE based on the conjectured intractability of the Module Learning with Error (MWLE) problem

ML-KEM.KeyGen()

1: $(\mathtt{pk}, \mathtt{sk}') \xleftarrow{\$} \mathtt{K\text{-}PKE.KeyGen()}$
2: $h \leftarrow \mathtt{SHA3\text{-}256}(\mathtt{pk})$
3: $z \xleftarrow{\$} \mathcal{M}$
4: $\mathtt{sk} \leftarrow (\mathtt{sk}', \mathtt{pk}, h, z)$
5: **return** $(\mathtt{pk}, \mathtt{sk})$

ML-KEM.Encap(pk)

1: $m \xleftarrow{\$} \mathcal{M}$
2: $h \leftarrow \mathtt{SHA3\text{-}256}(\mathtt{pk})$
3: $(K, r) \leftarrow \mathtt{SHA3\text{-}512}(m, h)$
4: $c \leftarrow \mathtt{K\text{-}PKE.Enc}(\mathtt{pk}, m, r)$
5: **return** (c, K)

ML-KEM.Decap(sk, c)

1: $\hat{m} \leftarrow \mathtt{K\text{-}PKE.Dec}(\mathtt{sk}', c)$
2: $(\overline{K}, \hat{r}) \leftarrow \mathtt{SHA3\text{-}512}(\hat{m}, h)$
3: $\hat{c} \leftarrow \mathtt{K\text{-}PKE.Enc}(\mathtt{pk}, m, \hat{r})$
4: **if** $\hat{c} = c$ **then**
5: $\quad K \leftarrow \overline{K}$
6: **else**
7: $\quad K \leftarrow \mathtt{SHA3\text{-}256}(c, z)$
8: **end if**
9: **return** K

Figure B.2: ML-KEM uses the $\mathtt{KEM}^{\not\perp}_{m}$ variant of the modular Fujisaki-Okamoto KEM transformation